

从0到1构建 AI Agent

原理、工具调用与企业级应用实践课程

上海成趣信息科技有限公司 杨晓春 2026-8-20

课程简介

- 课程时长：1天 / 6小时
- 课程形式：概念讲解 + 案例演示 + 动手实践
- 综合案例：企业知识库智能助手

课程学习目标

- 理解 AI Agent 的定义、特点和典型架构；
- 区分 Chatbot、Prompt 应用、RAG、工作流与 Agent；
- 构建一个能够调用工具的基础 Agent；
- 搭建企业知识库问答 Agent；
- 设计 Agent 工作流和多 Agent 协作方案；
- 识别 Agent 的安全风险；
- 对 Agent 进行测试、评估和部署。

模块	主题	时长 分钟	教学目标	主要内容	实践活动 / 阶段产出
一	AI Agent 认知与应用场景	30	建立对 AI Agent 的整体认识，理解其应用价值	1. AI Agent 的定义与特点 2. Agent 与聊天机器人、Prompt、RAG、自动化工作流的区别 3. Agent 的典型应用场景 4. 判断业务是否适合使用 Agent	分析出差助手、客服、数据分析等案例；能够判断业务需求是否适合采用 Agent
二	Agent 核心架构与运行机制	45	理解 Agent 的技术组成和基本运行循环	1. Agent 基本架构 2. 大语言模型、Prompt、工具、记忆、知识库等核心组件 3. Planning、ReAct、状态机等执行方式 4. Agent 的思考—行动—观察循环 5. Agent 设计原则	将“出差申请助手”拆解为用户输入、知识、工具、决策步骤和输出结果
三	构建第一个 AI Agent	75	使用 Python 和大模型 API 构建可运行的基础 Agent	1. API Key、模型参数、Token、Temperature 2. System Prompt 设计 3. 工具定义与调用 4. Agent 判断是否调用工具 5. 异常处理与结果生成6. LangChain、LangGraph 等框架简介	实现天气查询与计算器 Agent；完成能够理解问题、调用工具并生成回答的基础 Agent
四	RAG 知识库 Agent	60	掌握企业知识库问答的基本方法，降低模型幻觉	1. RAG 工作原理 2. 文档加载与切分 3. 文本向量化 4. 向量数据库与相似度检索 5. 检索结果与模型生成结合	基于企业制度或产品文档搭建知识库；实现文档检索、依据文档回答和来源展示
五	Agent 工作流、工具调用、MCP与多 Agent	60	掌握复杂任务的流程编排和多 Agent 协作方法	1. 固定工作流与 Agent 的区别 2. 顺序、分支、循环、并行、重试流程 3. 人工确认节点 4. 多 Agent 角色设计与协作 5. 单 Agent 与多 Agent 的选择原则	设计“智能会议助手”；实现会议纪要提取、任务分配、负责人识别和结果审核
六	Agent 评估、安全与部署	45	让 Agent 具备可靠、安全和可上线的基础条件	1. 效果、性能和用户体验指标 2. 测试集与评估方法 3. Prompt Injection、数据泄露、越权访问等风险4. 权限控制、人工确认、参数校验、日志记录 5. Streamlit、Gradio、FastAPI、Docker 等部署方式	为项目设计 5 条测试用例、3 条安全规则、1 套异常处理方案和上线检查清单

课程整体结构

模块	主题	产出成果	时长（分钟）
1	AI Agent认知与应用场景		30
2	Agent核心架构与运行机制		45
3	构建第一个AI Agent	天气与计算器Agent、企业制度问答Agent	75
4	RAG知识库Agent	企业制度知识库问答 Agent	60
5	Agent工作流、工具调用、MCP与多Agent	智能会议助手案例	60
6	Agent评估、安全与部署	测试用例	45
7	实战：综合项目与成果展示	企业智能知识助手	45

课程参考代码

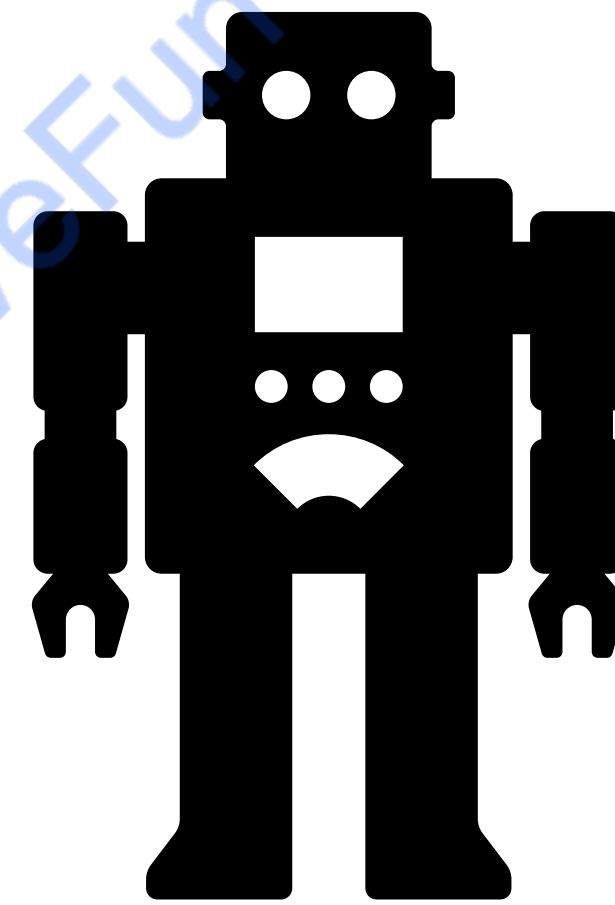
产出成果	参考代码链接
天气与计算器Agent	https://github.com/bettermorn/AIAgent/blob/main/code/ReActAgent/weather_calculator_agent.py
企业制度问答Agent	https://github.com/bettermorn/AIAgent/blob/main/code/ReActAgent/weather_calculator_agent.py
企业制度知识库问答	https://github.com/bettermorn/AIAgent/tree/main/code/KBAgent
智能会议助手案例	https://github.com/bettermorn/AIAgent/tree/main/code/MultiAgent/Meeting_Assistant
测试用例	https://github.com/bettermorn/AIAgent/blob/main/code/MultiAgent/Meeting_Assistant/Security_check.md
企业智能知识助手	https://github.com/yh-yao/super_agent_book/tree/main/RAG_%E6%99%BA%E8%83%BD%E5%AE%A2%E6%9C%8D%E4%B8%8E%E7%9F%A5%E8%AF%86%E9%97%AE%E7%AD%94 修改部分参考 https://github.com/bettermorn/AIAgent/tree/main/code/CSAIAgent

模块一： AI Agent认知与应用场景

建立对 AI Agent 的整体认识，理解 Agent 为什么成为大模型应用的重要方向。

什么是AI Agent?

- AI Agent 是能够围绕目标自主完成任务的智能系统。
- 核心能力
 - 理解用户目标；
 - 拆解复杂任务；
 - 制定执行计划；
 - 调用外部工具；
 - 获取环境反馈；
 - 根据反馈调整行动；
 - 输出最终结果。
- 大模型负责思考，工具负责执行，**Agent** 负责协调。



从聊天机器人到AI Agent

- Agent 不只是“回答问题”，而是能够：
 - 采取行动；
 - 使用外部系统；
 - 根据执行结果继续决策；
 - 完成多步骤任务。

普通聊天机器人： 用户提问 → 模型回答

AI Agent:

用户目标



理解任务



制定计划



调用工具



获取结果



调整计划



完成任务

AI Agent与相关概念的区别

类型	核心特点	典型场景
聊天机器人	对话问答	在线客服
Prompt 应用	固定模板生成内容	文案生成
RAG 应用	检索知识后回答	企业知识库
自动化 workflow	按固定流程执行	审批流程
AI Agent	动态决策、调用工具	智能助理
多 Agent	多角色协同完成任务	复杂项目分析

AI Agent的典型应用场景

• 企业办公

- 会议纪要与任务跟进;
- 邮件撰写与分类;
- 日程安排;
- 企业制度问答。

• 业务运营

- 销售助理;
- 客户服务;
- 市场调研;
- 产品推荐。

• 技术与数据

- 数据分析;
- 代码开发;
- IT 运维;
- 自动生成业务报告。

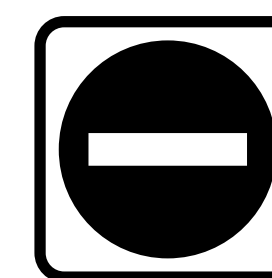
什么业务适合使用Agent?

• 适合使用Agent的场景



- 目标比较明确，但实现路径不固定；
- 需要调用多个工具或系统；
- 需要处理多步骤任务；
- 存在大量自然语言交互；
- 需要根据结果动态调整流程。

• 不一定适合使用Agent的场景



- 流程固定且规则明确；
- 对结果稳定性要求极高；
- 任务风险较高且无法人工审核；
- 使用简单脚本即可解决。

判断原则:能用固定流程解决的问题，不必强行使用 Agent。

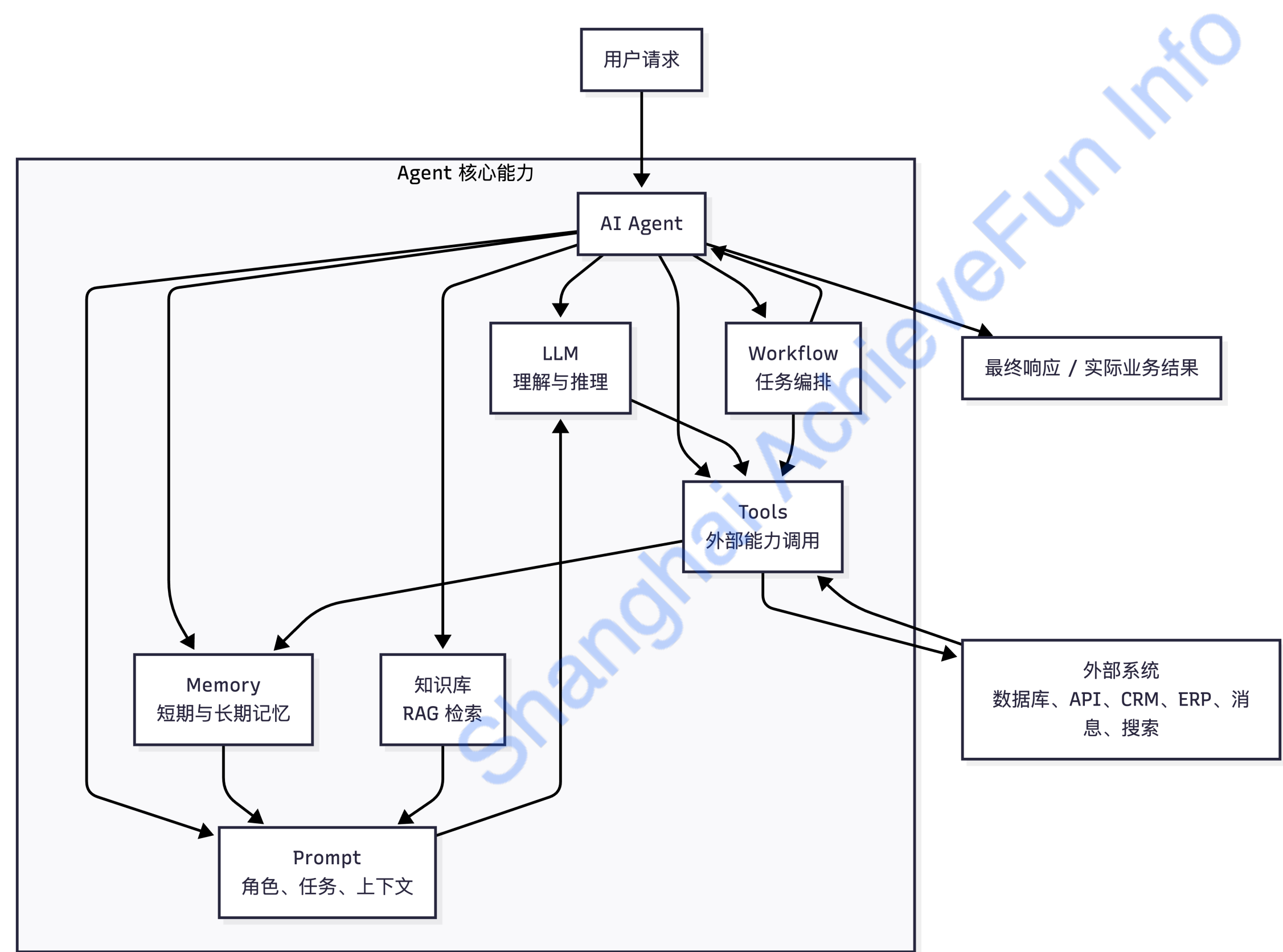
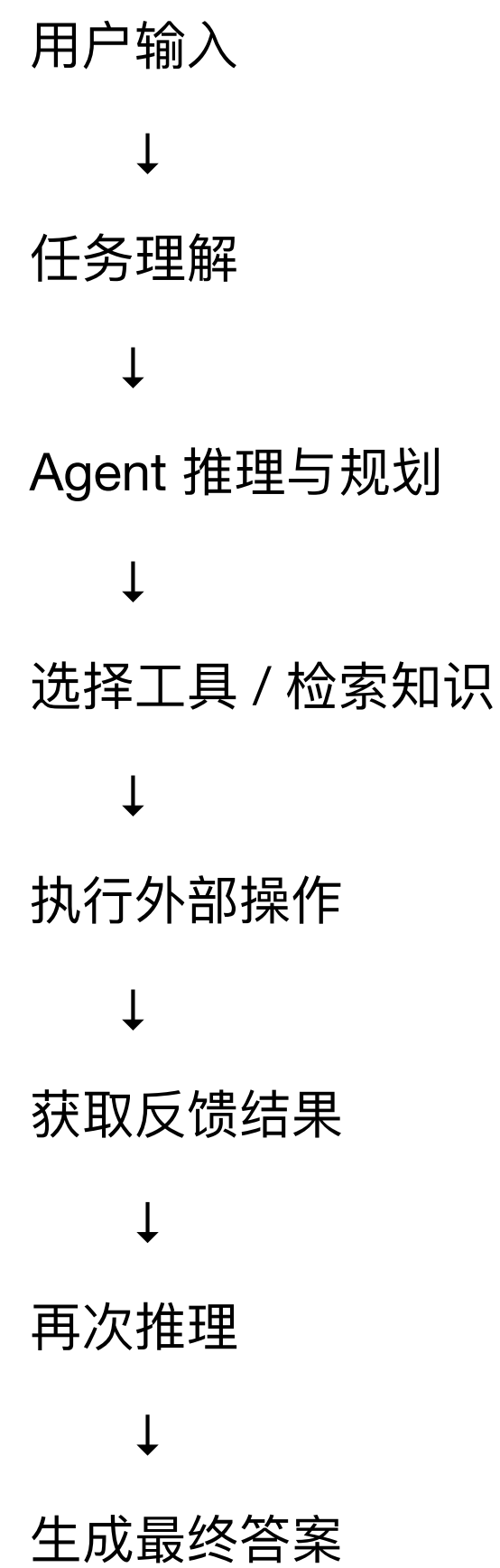
实践练习1： 判断一个业务需求是否适合采用Agent

任务	是否适合使用 Agent	更合适的实现方式
查询员工报销制度	部分适合	知识库问答或轻量 Agent
自动查询天气并安排出行	适合	多工具协同 Agent
分析销售数据并生成报告	适合	数据分析 Agent
自动审批所有财务申请	不适合完全自动化	规则引擎 + Agent 辅助 + 人工审批
根据客户需求推荐产品	适合	推荐型 Agent

模块二： AI Agent核心架构与运行机制

理解 Agent 的技术组成和基本运行循环，为后续实操建立基础。

Agent的基本结构和核心组成



- LLM：理解、推理和生成；
 - Prompt：定义角色、规则和边界；
 - Tools：执行外部操作；
 - Memory：保存上下文和用户偏好；
 - Knowledge Base：提供专业和企业知识；
 - Workflow：管理任务执行过程。
- 常见方式包括：
- ReAct：思考—行动—观察；
 - Plan-and-Execute：先规划，再执行；
 - 流程图式编排；
 - 状态机；
 - 多 Agent 协作。

大语言模型在Agent中的作用

- 大语言模型主要负责：
 - 理解用户意图；
 - 识别任务类型；
 - 分析上下文；
 - 选择合适工具；
 - 组织任务步骤；
 - 生成自然语言结果；
 - 处理异常和追问。
- **注意:** 大语言模型本身并不等于 Agent。模型是“大脑”，Agent 是完整的“智能系统”。

Prompt 的作用

- 一个好的 System Prompt 应包含

- Agent 身份;
- 服务目标;
- 工作步骤;
- 工具使用规则;
- 输出格式;
- 禁止行为;
- 异常处理;
- 安全要求。

- 示例

- 你是一名企业智能助手。请遵循以下规则:

1. 优先使用知识库回答企业制度问题;
2. 不确定时必须明确说明;
3. 不得编造业务数据;
4. 涉及付款、删除、审批等操作时,必须先获得用户确认;
5. 回答应简洁,并尽可能提供信息来源。

工具Tools是什么？

- 工具是 Agent 可以调用的外部能力。
- 常见工具
 - 计算器；
 - 搜索引擎；
 - 天气 API；
 - 数据库查询；
 - 文件读写；
 - 邮件发送；
 - 日历管理；
 - CRM、ERP、OA 系统。
- 工具定义需要包含
 - 工具名称；
 - 工具用途；
 - 输入参数；
 - 参数类型；
 - 返回结果；
 - 使用限制；
 - 权限要求。

Agent 的典型运行循环

- **ReAct**: 思考-行动-观察

Thought: 理解问题并判断下一步



Action: 调用工具或检索知识



Observation: 获取工具执行结果



Thought: 分析结果并决定是否继续



Final Answer: 生成最终答案

- 示例

用户: 北京今天适合出差吗?

1. 判断需要天气信息;
2. 调用天气工具;
3. 获取天气结果;
4. 结合出差场景进行建议;
5. 输出最终回答。

Agent的设计原则

1. 能用固定 workflow 解决，就不要过度使用 Agent；
2. 工具数量不宜过多；
3. 每个工具职责应当单一；
4. 工具名称和描述必须清晰；
5. 重要操作必须经过用户确认；
6. 高风险任务必须设置人工审核；
7. 对错误和异常进行明确处理；
8. 记录调用过程，便于调试和审计。

实践练习2:根据AI Agent基本结构，拆解出差申请助手

Agent 模块	主要内容
用户输入	出差人员、地点、时间、事由、交通、费用、项目等
所需知识	差旅制度、审批规则、组织架构、预算规则、日期规则、合规要求
可用工具	员工查询、政策查询、预算计算、项目校验、申请创建、审批提交、状态查询、通知工具
Agent 决策步骤	理解意图 → 提取信息 → 补齐字段 → 查政策 → 算预算 → 做合规检查 → 生成草稿 → 用户确认 → 创建并提交
最终输出	政策说明、预算结果、申请草稿、风险提醒、申请单号、审批状态

模块三： 构建第一个 AI Agent

通过动手实践，实现一个能够理解任务并调用工具的基础 Agent。

实践目标

- 本模块将构建一个基础 Agent，实现：
 - 接收用户自然语言问题；
 - 判断是否需要调用工具；
 - 调用天气查询工具；调用计算器工具；
 - 根据工具结果生成回答；
 - 处理简单异常。
- 实践案例：天气与费用计算助手 和
- 参考代码链接：
 - https://github.com/bettermorn/AIAgent/blob/main/code/ReActAgent/weather_calculator_agent.py
 - https://github.com/bettermorn/AIAgent/blob/main/code/enterprise_policy_agent.py

推荐技术栈

- 基础环境
 - Python 3.10+
 - VS Code 或 Jupyter Notebook
 - Git
 - 大模型 API
- 开发框架
 - LangChain;
 - LangGraph;
 - OpenAI 兼容 API
 - Streamlit 或 Gradio。
- 可选模型
 - 通义千问;
 - 智谱;
 - DeepSeek;
 - 豆包;
 - 企业内部大模型。

调用大模型API的基本流程

配置 API Key



选择模型



编写 System Prompt



传入用户消息



获取模型响应



处理与展示结果

- 需要关注的参数

- Model;

- Temperature;

- Max Tokens;

- Timeout;

- Retry;

- Streaming;

- Token 消耗。

代码说明

- 工具设计要点
 - 输入参数明确；
 - 返回结果简单；
 - 对异常进行处理；
 - 避免执行危险代码；
 - 生产环境不应直接使用不受限制的 eval()。

```
else:
    umbrella_advice = "今天降雨概率较低，通常不必特意带伞。"
```

```
# 根据温度给出建议
try:
    temperature_number = float(temperature)
except (ValueError, TypeError):
    temperature_number = 20

if temperature_number <= 5:
    clothing_advice = "天气较冷，建议做好保暖。"
```

```
elif temperature_number >= 32:
    clothing_advice = "天气较热，注意防暑降温。"
```

```
else:
    clothing_advice = "温度总体适中，正常着装即可。"
```

```
travel_advice = f"{umbrella_advice}{clothing_advice}"

result = {
    "城市": city,
    "日期": today.get("date", "今天"),
    "当前天气": weather_description,
    "当前温度": f"{temperature}°C",
    "体感温度": f"{feels_like}°C",
    "今日最高温": f"{max_temp}°C",
    "今日最低温": f"{min_temp}°C",
    "湿度": f"{humidity}%",
    "风速": f"{wind_speed} km/h",
    "风向": wind_direction,
    "今日最大降雨概率": f"{max_rain_chance}%",
    "出行建议": travel_advice,
}

return json.dumps(
    result,
    ensure_ascii=False,
    indent=2,
)

except (KeyError, IndexError, TypeError) as e:
    return f"天气数据解析失败: {e}"
```

```
# =====
# 1. 天气工具
# =====

@tool
def get_weather(city: str) -> str:
    """
    查询指定城市今天的天气。

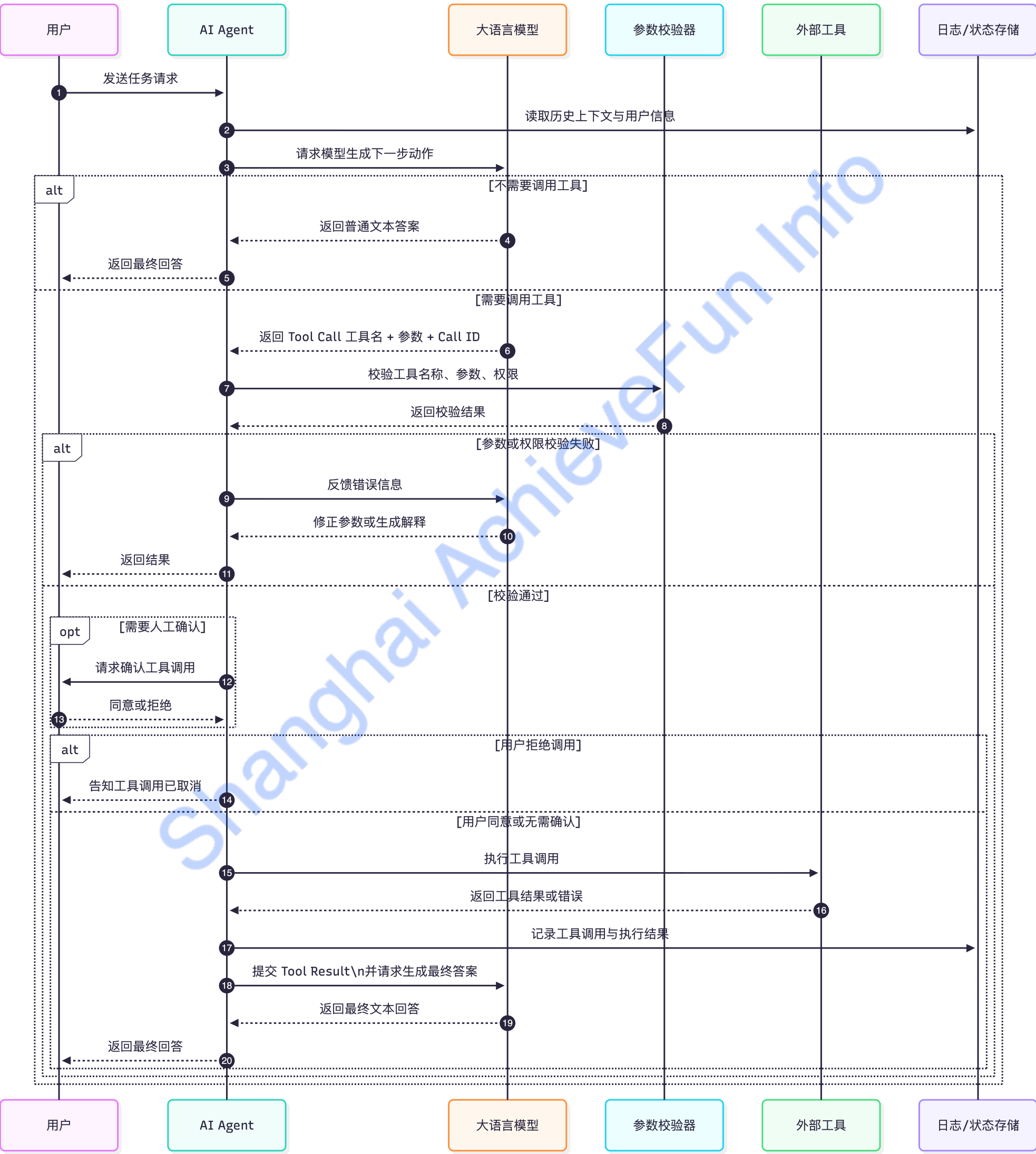
    参数:
    city: 城市名称，例如北京、上海、广州、深圳。

    返回:
    当前温度、体感温度、天气状况、降雨概率、风速以及出行建议。
    当用户询问天气、是否需要带伞、是否适合出差时，必须调用此工具。
    """

    city = city.strip()

    city_mapping = {
        "北京": "Beijing",
        "上海": "Shanghai",
        "广州": "Guangzhou",
        "深圳": "Shenzhen",
        "杭州": "Hangzhou",
        "南京": "Nanjing",
        "成都": "Chengdu",
        "重庆": "Chongqing",
        "武汉": "Wuhan",
        "西安": "Xi'an",
        "天津": "Tianjin",
        "苏州": "Suzhou",
        "厦门": "Xiamen",
```

AI Agent调用工具的流程



工具调用示例

用户问题

北京天气怎么样？如果我出差两天，机票1280 元，酒店每天 860 元，总费用是多少？

Agent 执行过程

- 1. 调用天气工具查询北京天气；
- 2. 调用计算器计算：
- 3. $1280 + 860 \times 2 = 3000$ 元 结合天气和费用信息；
- 4. 输出结构化结果。

最终答案示例

- 北京天气：晴，26℃；
- 两天住宿及机票费用：3000元；
- 建议：天气较好，适合出行。

```
# 5. 创建 LangChain Agent
# =====

tools = [
    get_weather,
    calculate,
]

SYSTEM_PROMPT = """
你是一个中文智能助手，负责处理天气查询和数学计算。

你的行为规则：

1. 用户询问天气时，必须调用 get_weather 工具，不能凭空编造天气。
2. 用户询问“是否需要带伞”时，必须先调用 get_weather 工具。
3. 用户询问“是否适合出差”时，必须先查询天气，再结合温度、降雨概率、风力给出建议。
4. 用户询问总费用、价格相加或其他数学问题时，必须调用 calculate 工具。
5. 计算结果必须以 calculate 工具返回的结果为准，不能自行猜测。
6. 最终回答使用中文，表达清晰、简洁。
7. 如果是天气问题，请说明天气数据可能存在实时延迟。
8. 如果用户的问题同时包含多个任务，需要依次调用相应工具。
9. 如果工具返回错误信息，应当如实告知用户，不要编造结果。

天气建议可以参考：

- 降雨概率较高：建议带伞；
- 温度过低：建议保暖；
- 温度过高：注意防暑；
- 风力较大：出差或户外活动需要谨慎。
"""

agent = create_agent(
    model=llm,
    tools=tools,
    system_prompt=SYSTEM_PROMPT,
    middleware=[
        log_before_model,
        log_after_model,
        handle_tool_errors,
    ],
)
```

实践任务3.1:完成一个天气与计算器Agent

- 任务要求
 - 至少配置两个工具；
 - 支持自然语言提问；
 - 能自动判断是否调用工具；
 - 返回结构化结果；
 - 对错误输入进行提示。
- 参考代码：https://github.com/bettermorn/AIAgent/blob/main/code/ReActAgent/weather_calculator_agent.py
- 测试问题
 - 北京今天适合出差吗？
 - 上海天气怎么样？
 - 机票 1280 元、酒店 860 元，总费用是多少？
 - 查询不存在的城市。

实践任务3.2:完成一个企业制度问答Agent

给 Agent 配置几条制度内容：

- 出差报销标准；
- 请假制度；
- 加班申请流程；
- 采购审批规则。

要求 Agent：

- 先判断问题类型；
- 调用对应工具或知识；
- 给出简洁回答；
- 对没有依据的信息不进行猜测。

- 测试问题

- 去成都出差，住宿每天最多可以报销多少？
- 请假两天需要谁审批？
- 加班应该提前多久申请？
- 采购 8 万元的设备需要哪些审批？
- 公司员工迟到一次扣多少钱？

- 参考代码

- https://github.com/bettermorn/AIAgent/blob/main/code/ReActAgent/weather_calculator_agent.py

模块四：RAG知识库Agent

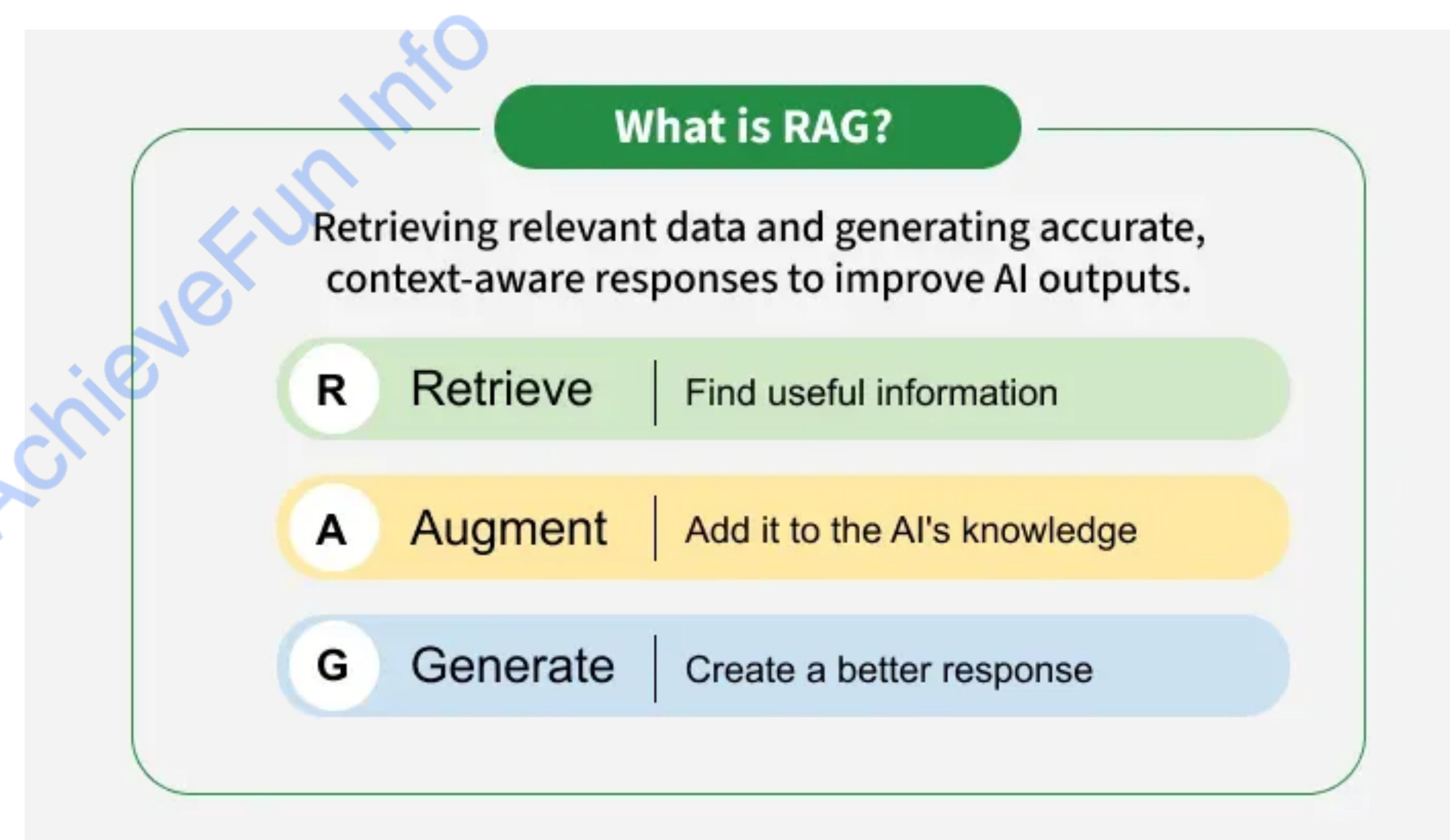
理解并实践企业知识库问答，解决大模型“不了解企业内部知识”和“容易产生幻觉”的问题。

为什么需要知识库？

- 大语言模型存在以下局限：
 - 不知道企业内部信息；
 - 训练数据可能不是最新的；
 - 无法直接访问私有文档；
 - 可能产生幻觉；
 - 难以给出准确来源。
- 解决方案：通过 RAG 将企业知识提供给大模型。

什么是RAG?

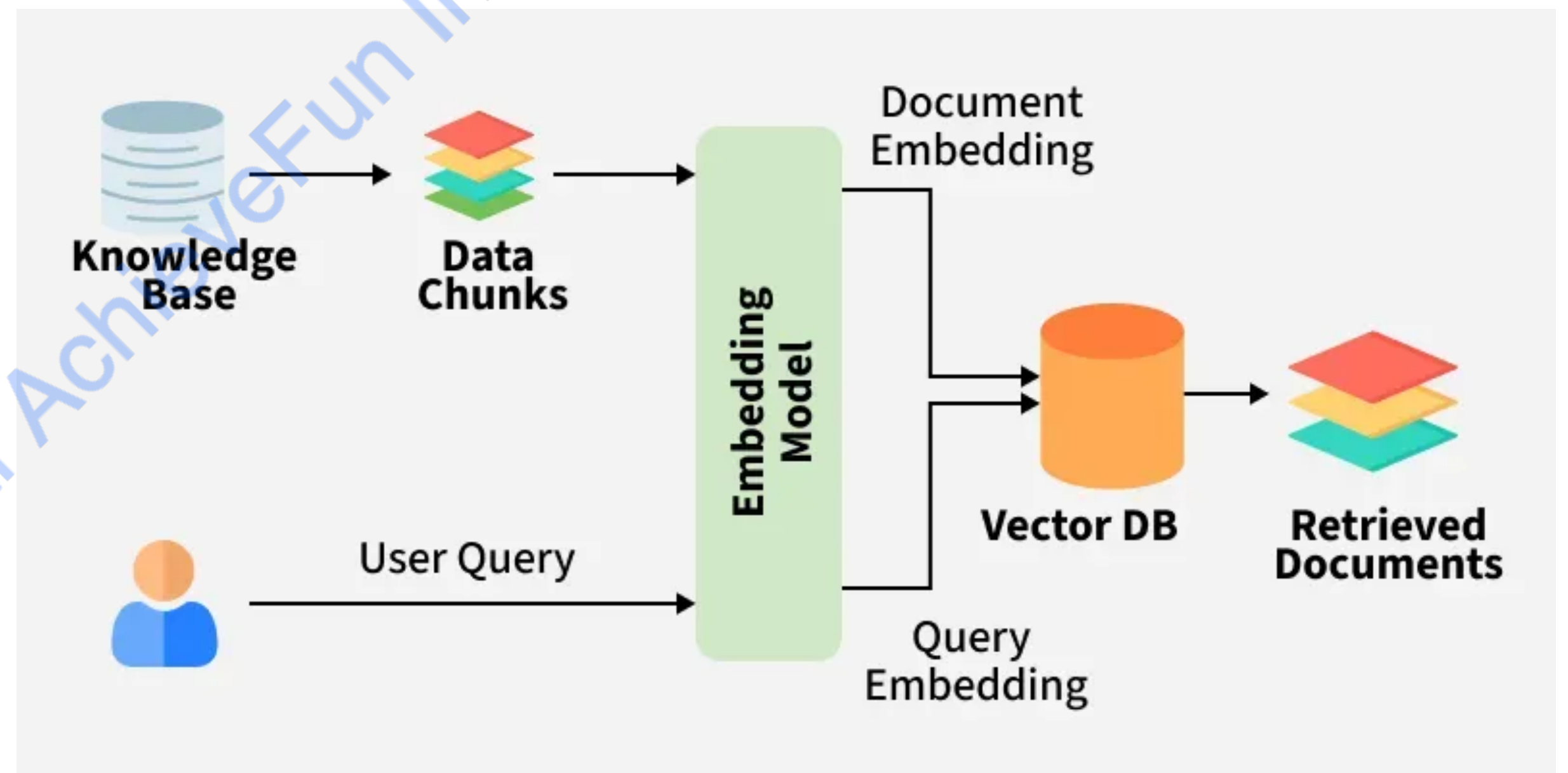
- 检索增强生成。
- 用户问题 ➡ 问题向量化 ➡ 检索相关知识 ➡ 拼接上下文 ➡ 大模型生成答案
- RAG 的核心思想：先查资料，再回答问题。



图片来源: <https://www.geeksforgeeks.org/nlp/what-is-retrieval-augmented-generation-rag/>

RAG的完整流程

- 文档导入 ➡ 文档解析 ➡ 文本切分 ➡ 向量化 ➡ 写入向量数据库 ➡ 用户提问 ➡ 相似度检索 ➡ 大模型生成回答
- RAG支持的知识来源: PDF; Word; Excel; Markdown; 网页; 数据库; 企业网盘; 产品说明书。



图片来源: <https://www.geeksforgeeks.org/nlp/what-is-retrieval-augmented-generation-rag/>

RAG的关键环节

1. 文档切分

- 切分过大：
 - 检索不够精准；
 - 上下文过长；
 - 增加模型成本。
- 切分过小：
 - 语义不完整；
 - 上下文缺失；
 - 难以准确回答。

2. 向量化

将文本转换为向量，用于相似度匹配。

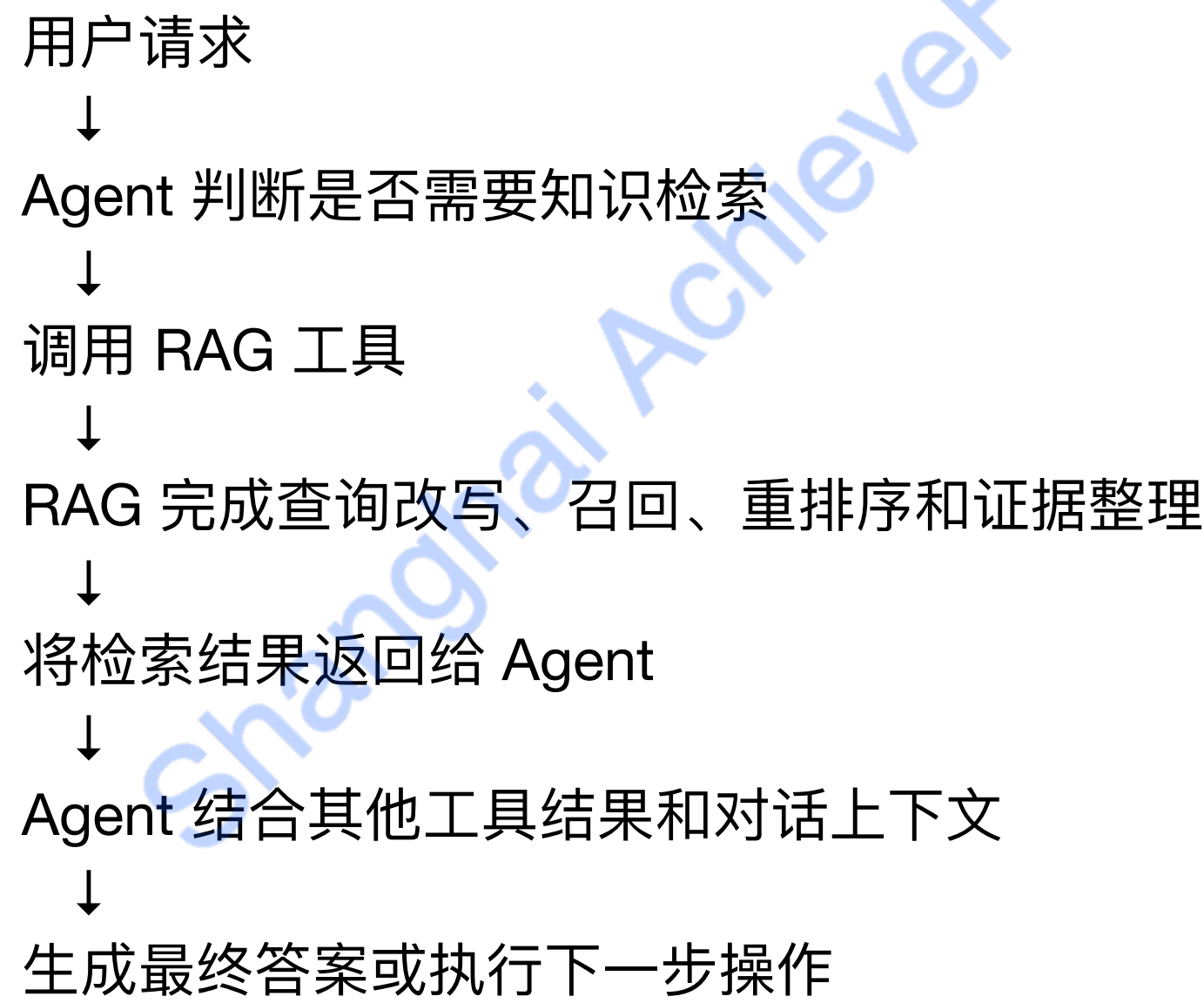
3. 向量数据库

常见工具：

- FAISS；
- Chroma；
- Milvus；
- Qdrant；
- Elasticsearch；
- pgvector。

RAG与Agent的关系

Agent 不一定直接执行检索和生成，而是把 RAG 封装成一个可调用的工具。当 Agent 判断用户问题需要查询外部知识时，就调用 RAG 工具，并将检索结果用于最终回答。

- RAG是Agent的知识获取工具，负责：
 - 查找相关知识；
 - 返回原文片段；
 - 提供文档来源；
 - 提供相关性评分；
 - 对结果进行过滤和排序。
- 

```
graph TD; A[用户请求] --> B[Agent 判断是否需要知识检索]; B --> C[调用 RAG 工具]; C --> D[RAG 完成查询改写、召回、重排序和证据整理]; D --> E[将检索结果返回给 Agent]; E --> F[Agent 结合其他工具结果和对话上下文]; F --> G[生成最终答案或执行下一步操作];
```
- Agent是负责决策、规划和协调工具的控制中心。负责：
 - 判断是否需要调用 RAG；
 - 决定检索什么内容；
 - 组合多个工具；
 - 根据检索结果继续规划；
 - 生成最终回答或执行后续动作。

企业制度问答示例

- 用户问题
 - 出差住宿费用可以报销多少
- Agent 流程
 1. 判断问题属于企业制度；
 2. 调用知识库检索；
 3. 找到“差旅报销制度”相关内容；
 4. 提取适用条件和金额标准；
 5. 给出答案及文档来源。

- 重要原则
 - 必须依据知识库回答；
 - 找不到内容时明确说明；
 - 不得自行补充制度；
 - 注意制度版本和适用范围。

RAG常见问题、原因和优化方向

问题	可能原因	优化方向
检索不到内容	文档未导入或切分不合理	检查数据和切分策略
检索结果不相关	查询表达与文档不同	使用混合检索或重写问题
回答不准确	上下文不足	调整召回数量
表格信息丢失	文档解析效果差	优化表格解析
答案过时	文档版本混乱	增加版本和有效期
模型编造答案	Prompt 约束不足	强制依据检索内容回答

实践任务4:基于企业制度文档，完成知识库问答Agent

- 任务步骤

1. 导入 PDF 或 Markdown 文档；
2. 对文档进行切分；
3. 创建向量索引；
4. 实现相似度检索；
5. 配置知识库检索工具；生成基于文档的回答；
6. 展示答案来源。

- 参考代码

<https://github.com/bettermorn/AI-Agent/tree/main/code/KBAgent>

- 交付结果

- 文档知识库；
- 检索功能；
- 问答 Agent；
- 3条测试问题；
- 参考来源展示。

运行RAG-QA-Agent

共加载 4 条 FAQ 数据
切分后得到 4 个文本块
FAISS 向量库已保存到：faiss_index_dashscope

检索结果：
1. 问题：如何申请年假？
答案：通过HR系统提交休假申请，主管审批后生效。

2. 问题：公司的工作时间是什么？
答案：工作日为周一至周五，上午9:00到下午6:00。

✅ 回答：通过HR系统提交休假申请，主管审批后生效。

📎 参考来源：
[1] 通过HR系统提交休假申请，主管审批后生效。
[2] 工作日为周一至周五，上午9:00到下午6:00。

✅ 回答：上午9:00

📎 参考来源：
[1] 工作日为周一至周五，上午9:00到下午6:00。
[2] 支持混合办公模式，每周可在家工作最多两天。

✅ 回答：可以，支持混合办公模式，每周可在家工作最多两天。

📎 参考来源：
[1] 支持混合办公模式，每周可在家工作最多两天。
[2] 通过HR系统提交休假申请，主管审批后生效。

✅ 回答：每周可在家工作最多两天。

📎 参考来源：
[1] 支持混合办公模式，每周可在家工作最多两天。
[2] 工作日为周一至周五，上午9:00到下午6:00。

✅ 回答：不可以，每周最多在家工作两天。

📎 参考来源：
[1] 支持混合办公模式，每周可在家工作最多两天。
[2] 工作日为周一至周五，上午9:00到下午6:00。

* Running on local URL: http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.

欢迎使用 RAG 问答系统！请问有什么可以帮助您的？

可以问我关于 HR 方面的问题

Chatbot

如何申请年假？

✅ 回答：通过HR系统提交休假申请，主管审批后生效。

📎 参考来源：
[1] 通过HR系统提交休假申请，主管审批后生效。
[2] 工作日为周一至周五，上午9:00到下午6:00。

提交

Use via API · Built with Gradio · Settings

模块五：Agent workflows、工具调用、MCP 与多 Agent

理解复杂任务的编排方式，能够判断什么时候使用 workflow、单 Agent 或多 Agent。

固定 workflows 与 Agent

- 固定 workflows

- 步骤1 → 步骤2 → 步骤3 → 步骤4

- 适合：

- 流程明确；
 - 步骤固定；
 - 风险较高；
 - 需要稳定和可审计。

- Agent

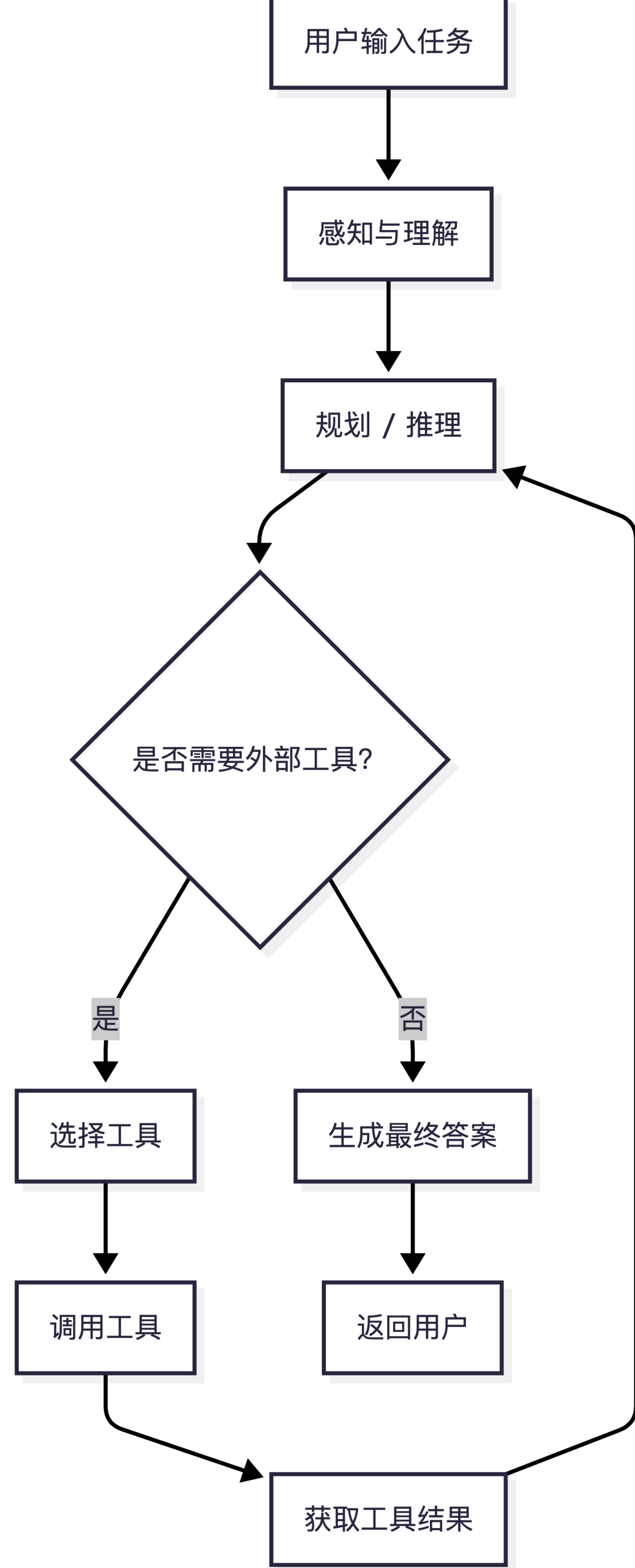
- 目标 → 动态规划 → 选择路径 → 执行
→ 调整

- 适合：

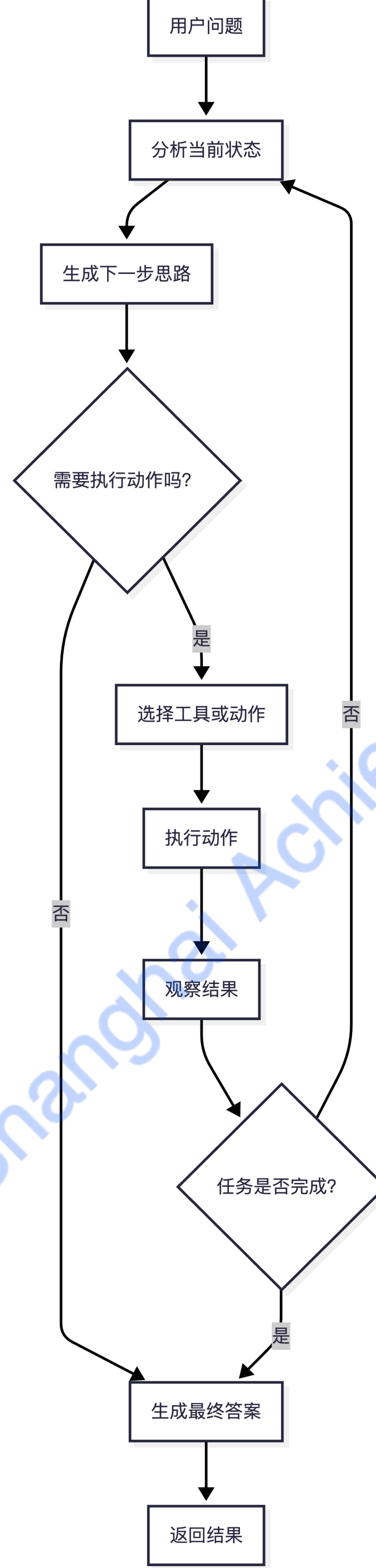
- 路径不固定；
 - 工具较多；
 - 任务需要动态判断；
 - 用户需求不完全明确。

常见Agent workflows模式

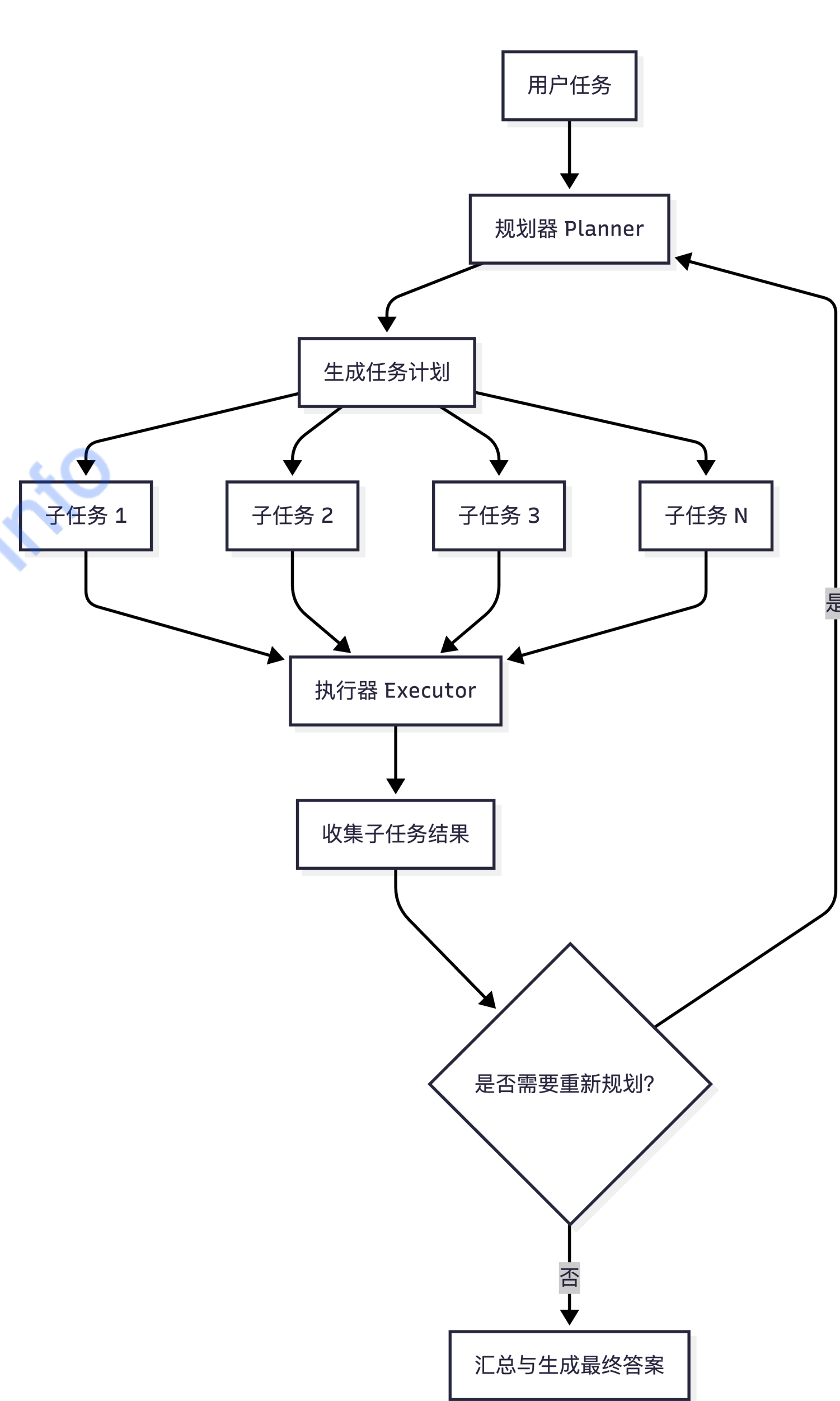
- 顺序执行；
 - 条件分支；
 - 循环执行；
 - 并行执行；
 - 人工确认；
 - 失败重试；
 - 子任务拆解；
 - 状态机。
- 可参考 <https://github.com/bettermorn/AI-Agent/blob/main/docs/AI-AgentConcept.md#%E5%B8%B8%E8%A7%81agent%E5%B7%A5%E4%BD%9C%E6%B5%81%E6%A8%A1%E5%BC%8F>



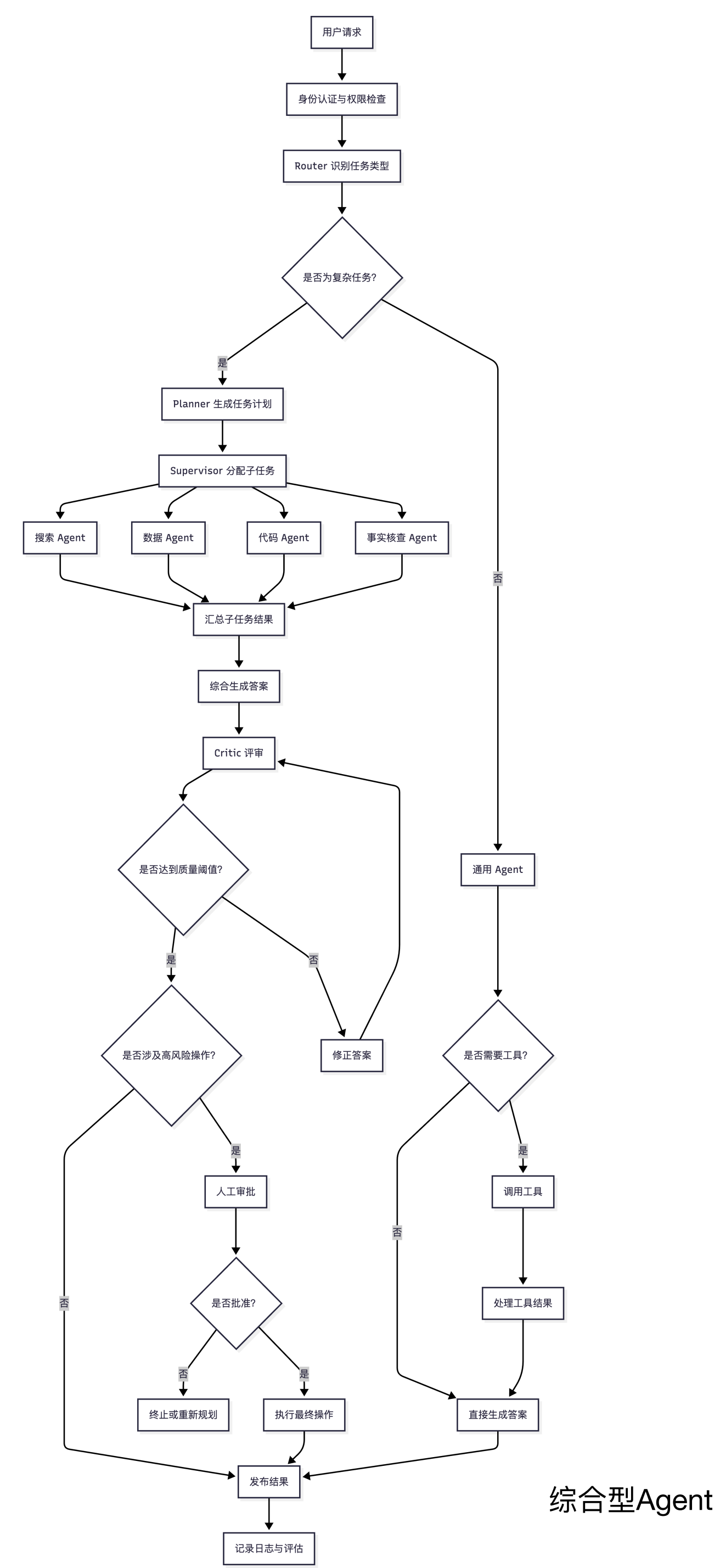
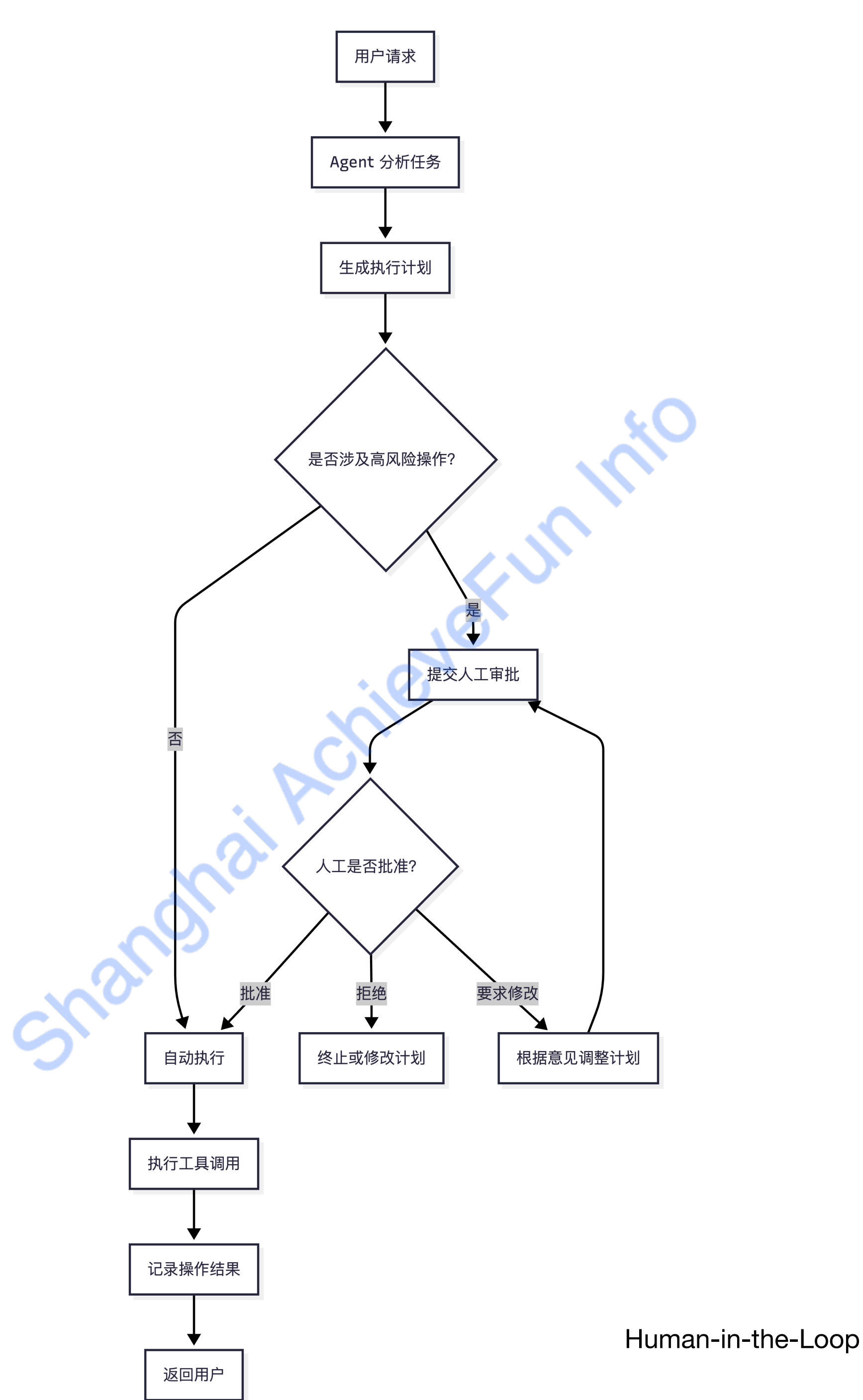
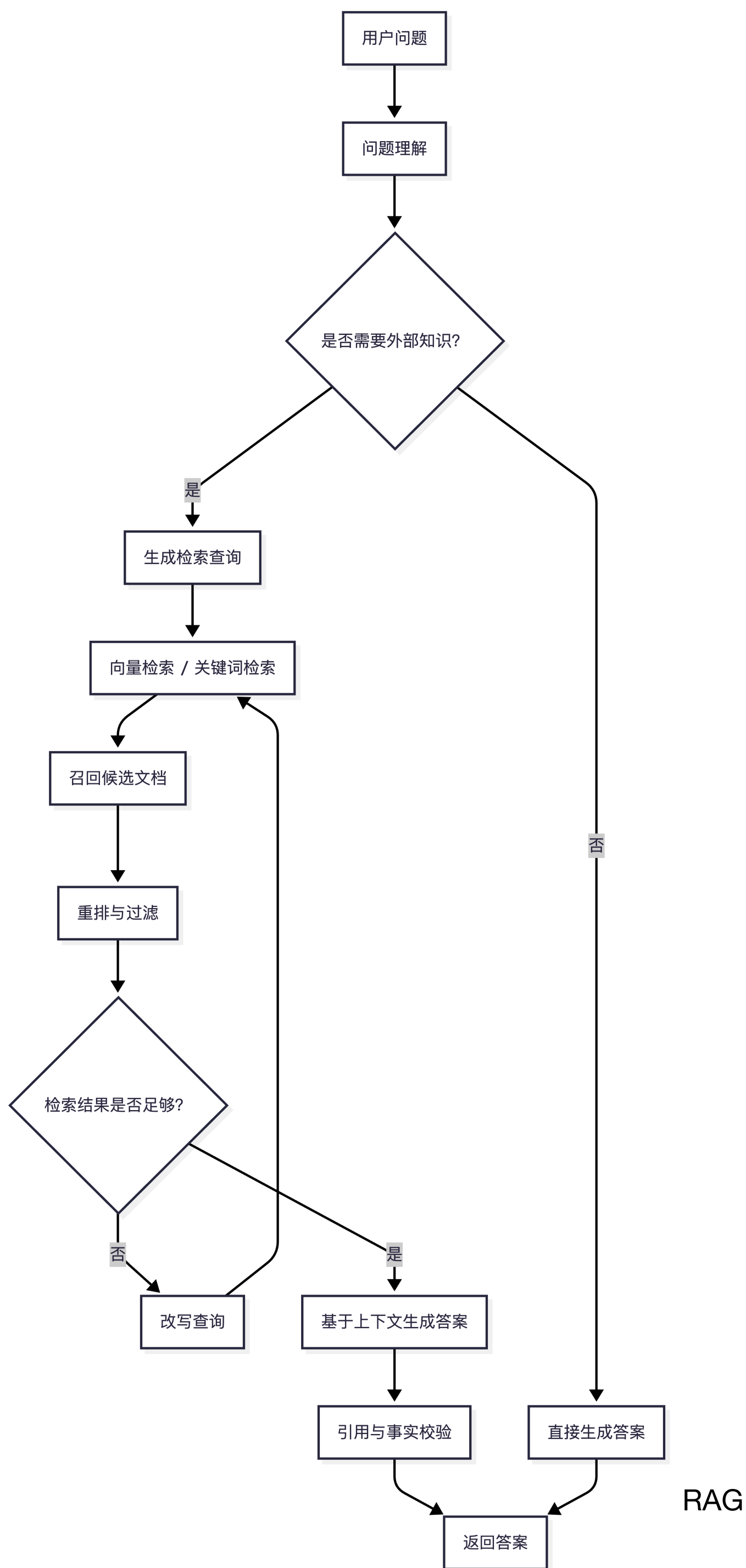
Basic



ReAct



Plan and Execute



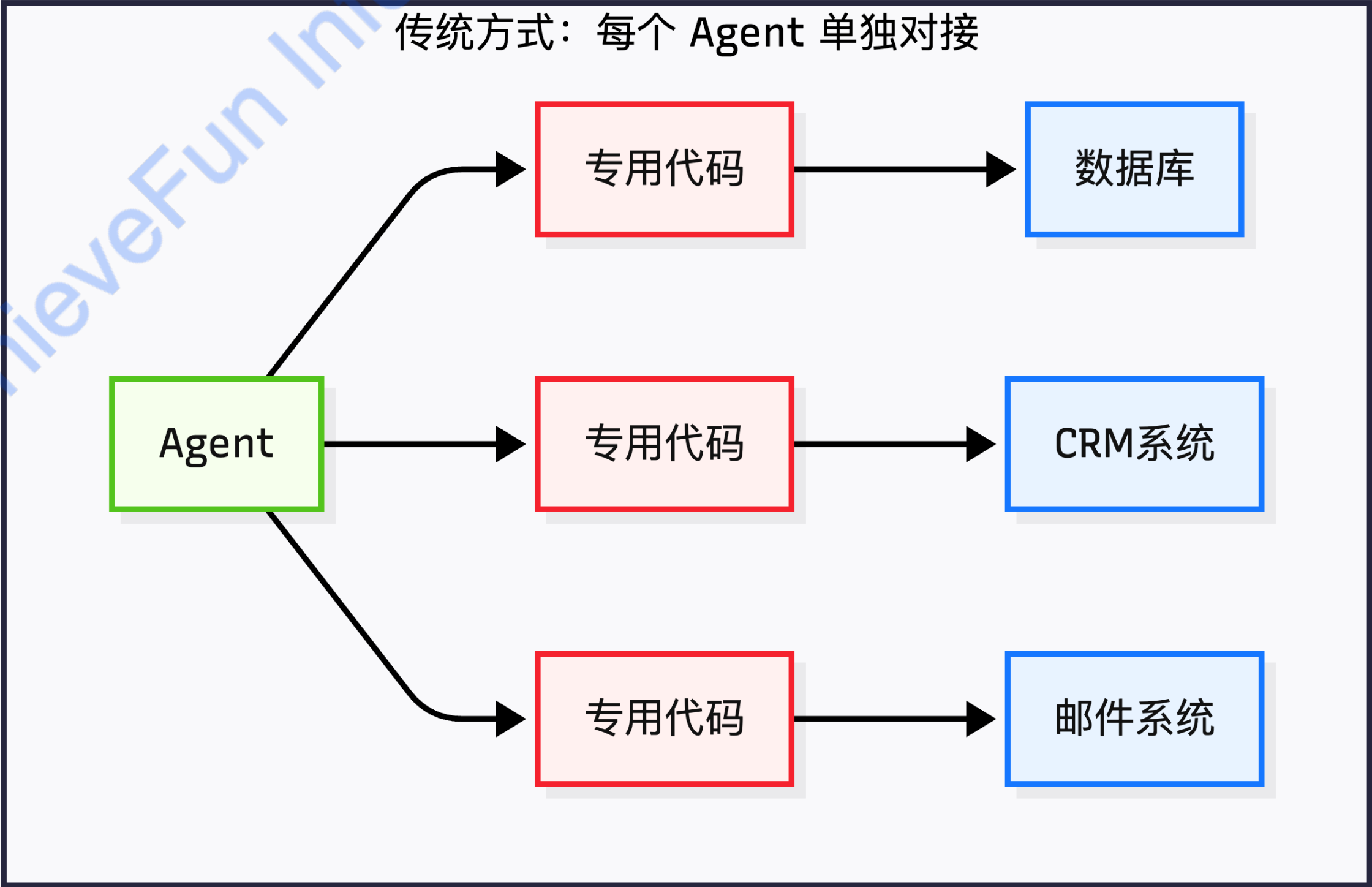
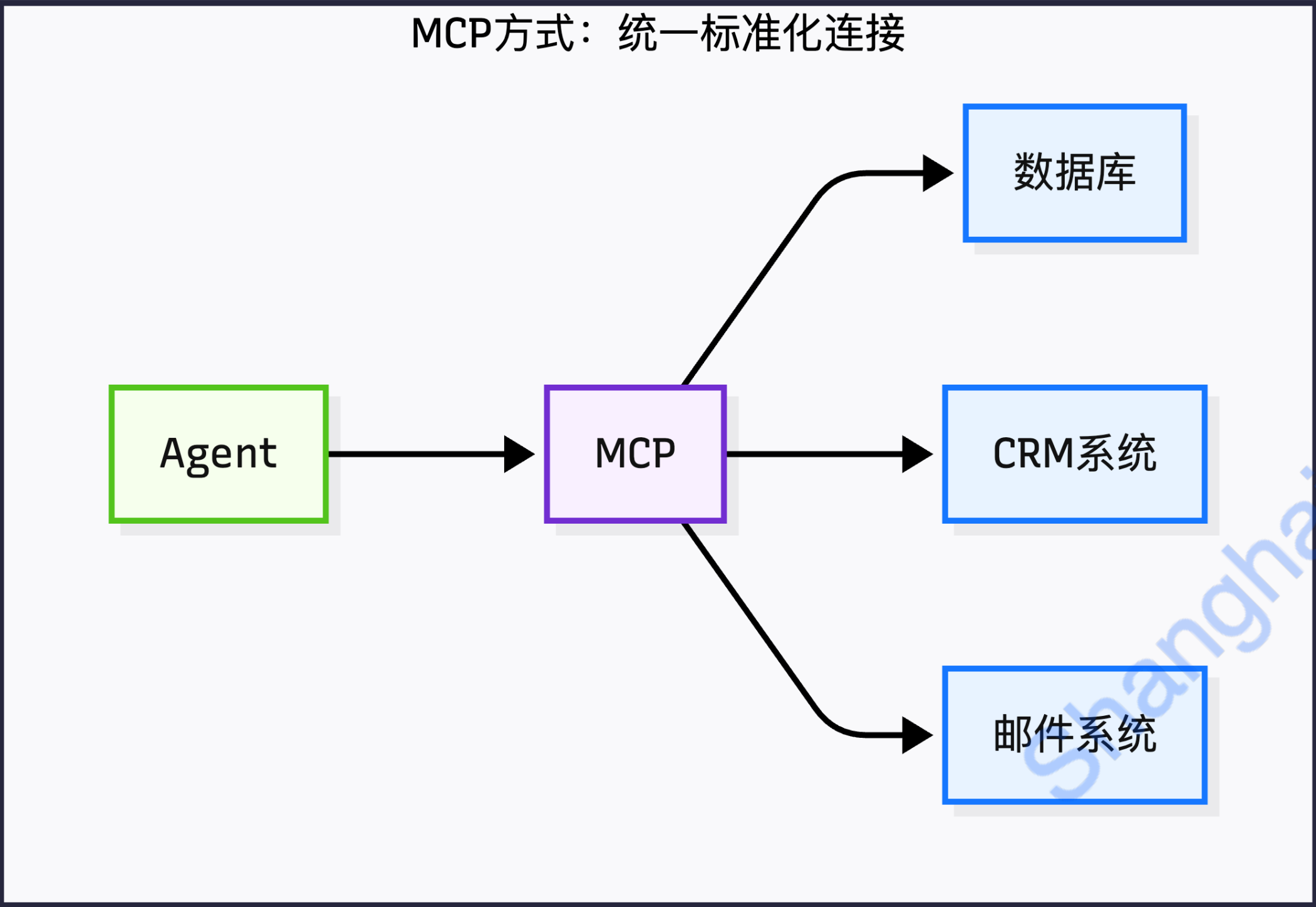
常见Agent workflows 模式对比

模式	核心特点	适合场景
基础 Agent 循环	感知、推理、行动循环	简单工具调用
ReAct	推理与行动交替	搜索、浏览器、动态决策
Plan-and-Execute	先规划再执行	复杂多步骤任务
Router	按任务类型分发	多领域、多业务入口
并行 Agent	多个 Agent 同时工作	独立子任务、批处理
Reflection	生成后自我评审和修正	写作、代码、报告
Supervisor	主管调度多个专家	多 Agent 协作
Human-in-the-Loop	关键节点人工确认	高风险、不可逆操作
RAG Agent	检索外部知识后生成	企业知识库、文档问答
异常恢复	重试、降级、人工介入	生产级工具调用

MCP (Model Context Protocol) : 连接 Agent 与外部能力的标准

- MCP是一套用于连接以下对象的开放协议：
 - AI应用
 - AI Agent
 - 外部工具
 - 数据资源
 - 企业业务系统
- MCP的主要价值
 - 标准化接入
 - 工具能力复用
 - 外部能力发现
 - 降低系统耦合
 - 方便扩展和维护
- MCP可以理解为 AI 应用连接外部世界的通用接口标准。Agent —— MCP —— 数据库、CRM、邮件、文件和业务系统

MCP与传统方式的对比



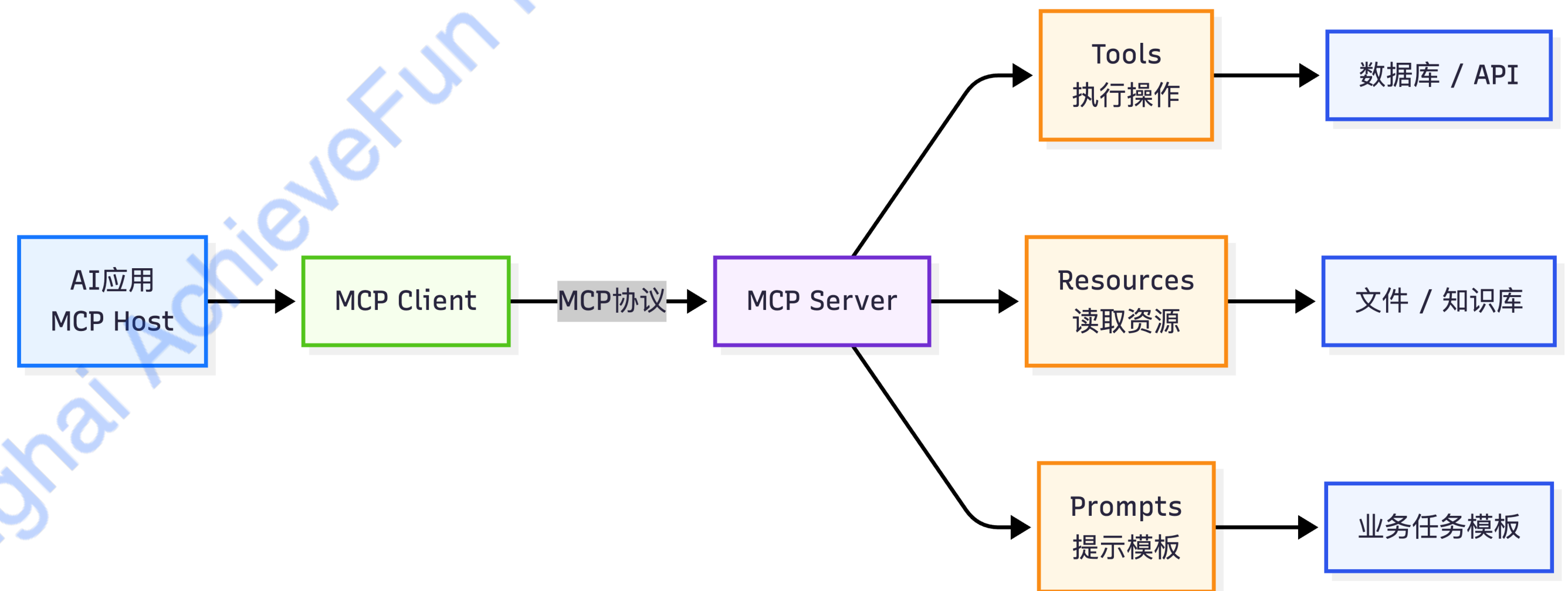
MCP如何工作？

- 核心角色

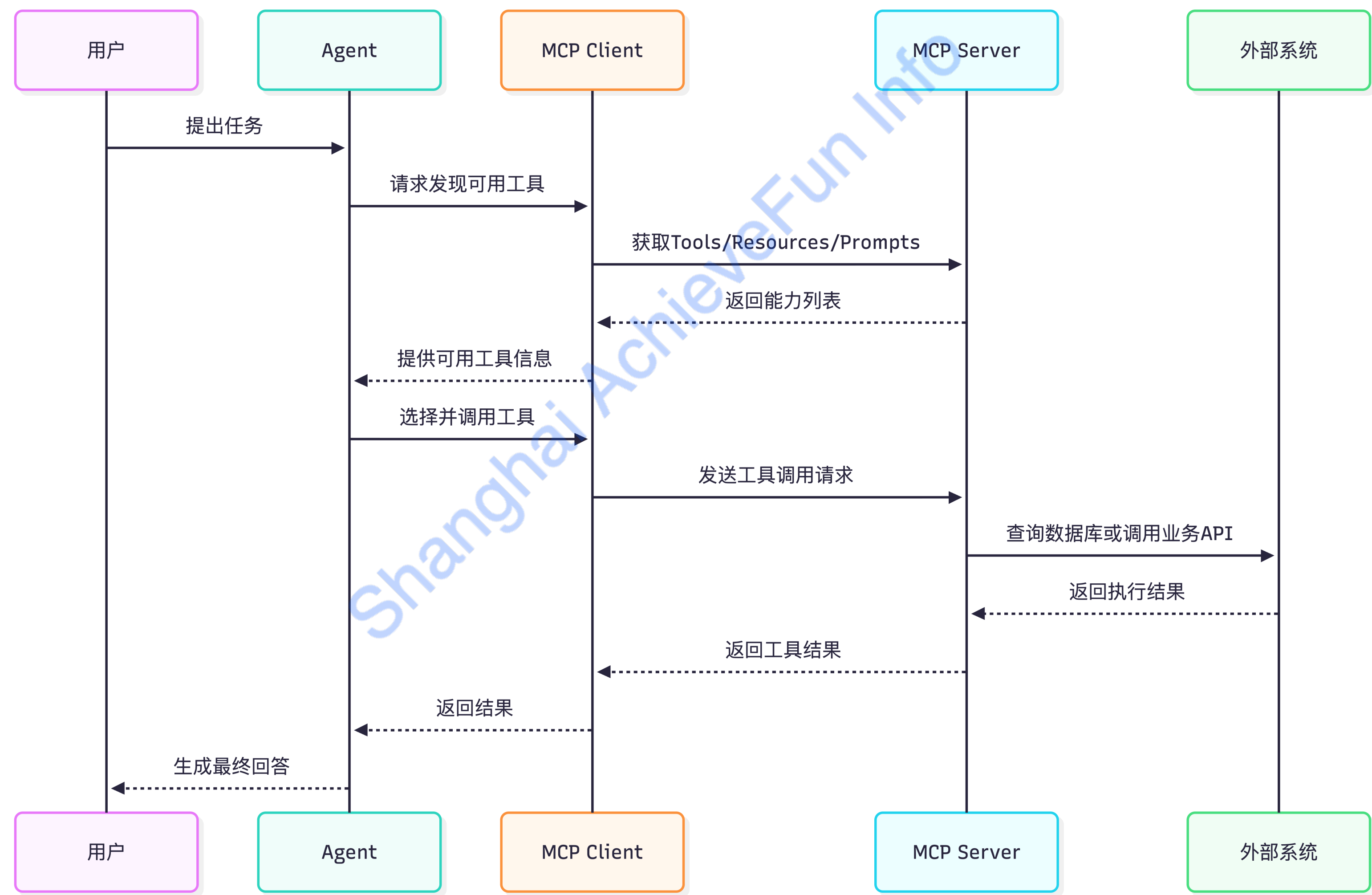
- MCP Host，承载 Agent 或 AI 应用
- MCP Client，负责连接和调用 MCP Server
- MCP Server，对外提供工具、数据和提示模板

- 核心能力

- Tools：工具，执行操作
- Resources：资源，提供可读取的信息
- Prompts：提示模板，提供可复用的任务模板



MCP调用流程



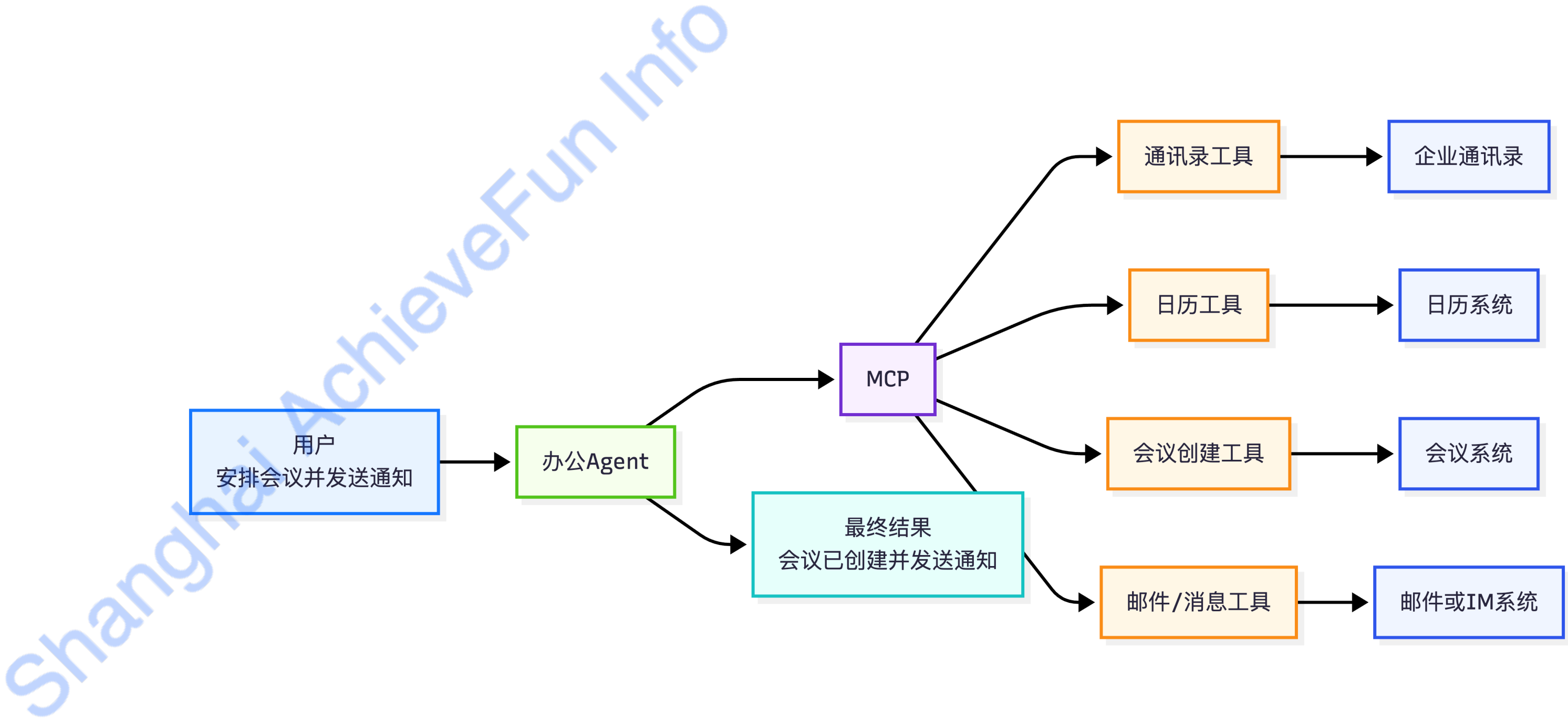
MCP、RAG、API和Function Calling有什么区别？

技术	主要解决的问题	典型作用
Agent	如何理解、规划并完成任务	分解任务、选择工具、执行流程
MCP	如何标准化连接外部能力	连接工具、数据源和业务系统
RAG	如何检索和利用外部知识	从文档或知识库中查找答案
API	系统之间如何通信	查询数据、提交请求、执行服务
Function Calling	模型如何请求调用函数	让模型选择并生成函数参数
工作流	如何控制任务执行步骤	编排固定或半固定的业务流程

- 用户：查询公司的报销制度，并判断我的报销是否符合要求
- RAG：检索报销制度
- MCP：连接知识库、财务系统或报销系统
- Agent：理解问题、组合信息并给出判断
- Function Calling：发起具体工具调用

MCP的应用价值和使用边界

- 使用 MCP 时的安全原则
 - 遵循最小权限原则
 - 区分读操作和写操作
 - 高风险操作增加人工确认
 - 校验工具输入参数
 - 对敏感数据进行脱敏
 - 记录工具调用和操作日志
 - 谨慎使用来源不明的 MCP Server
 - 对删除、退款、发信、发布等操作进行审批或二次确认



Agent负责思考，MCP负责连接，工具负责执行，RAG负责提供知识。

MCP Server的例子和应用

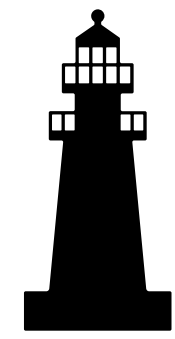
MCP Server类型	典型能力	适用场景
Filesystem	读取和搜索指定文件	文档分析、知识助手
Git/GitHub	查询代码、提交和Issue	编程Agent
PostgreSQL/MySQL	执行只读SQL、查询数据	数据分析、经营分析
Elasticsearch/OpenSearch	关键词和全文检索	企业搜索、RAG
Fetch/Playwright	获取网页、自动化浏览器	网络调研、流程自动化
Slack/Teams	查询消息、发送通知	协同办公
Google Drive/SharePoint	访问企业文档	企业知识助手
Jira/ServiceNow	查询或创建工单	研发和IT运维
Notion/Confluence	查询知识库页面	制度和项目资料问答

多Agent协作架构

- 典型角色
- 需求分析 Agent;
- 搜索 Agent;
- 数据分析 Agent;
- 内容生成 Agent;
- 审核 Agent;
- 项目管理 Agent。



多Agent的常见问题



- 角色之间沟通成本高；
- 任务重复执行；
- 状态难以管理；
- 错误会被逐步放大；
- Token 成本增加；
- 调试困难。
- 设计时应遵循： **优先使用单 Agent**，只有在职责、权限或任务明显分离时才使用多 Agent。

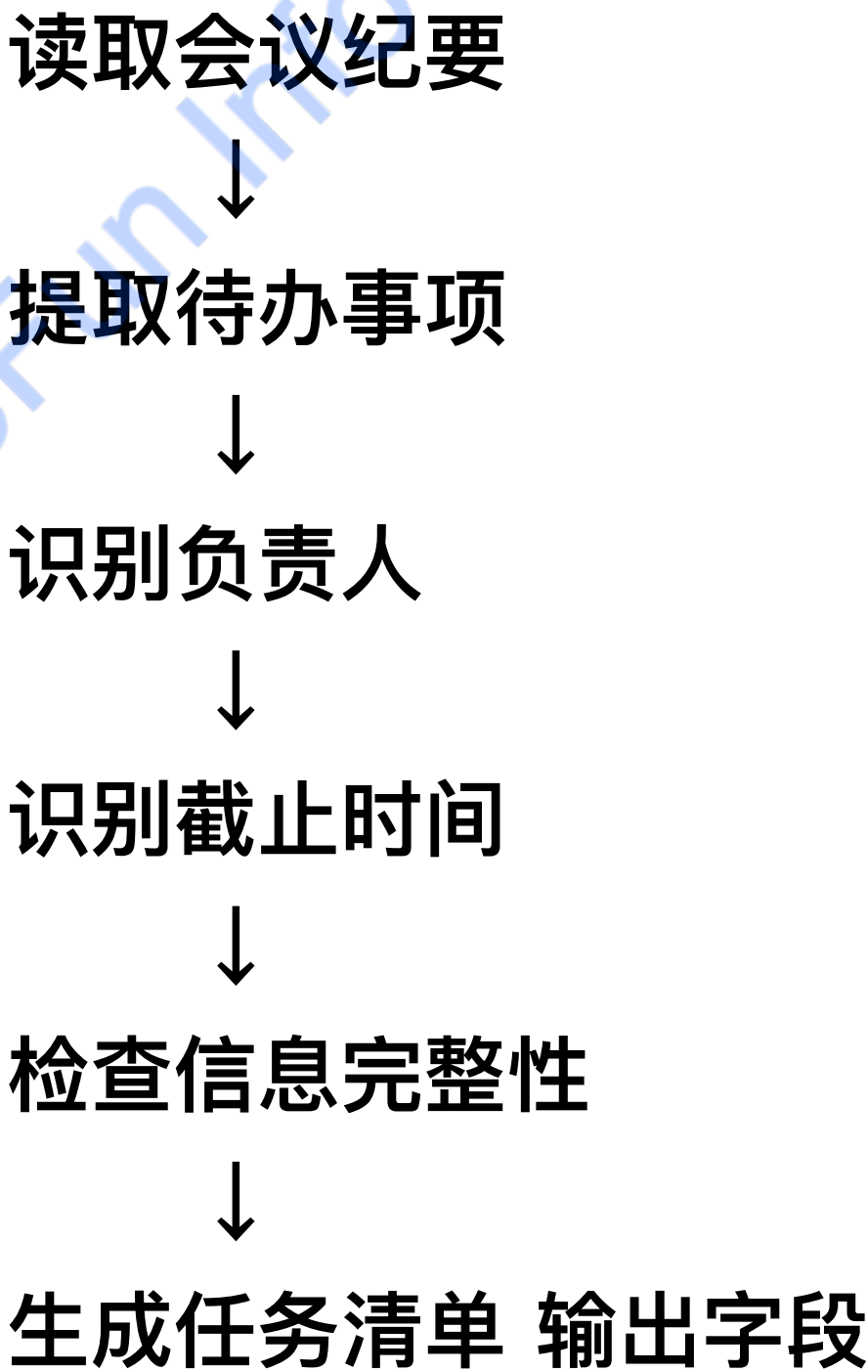
单Agent还是多Agent?

- 优先选择单 Agent
- 当：
 - 任务规模较小；
 - 工具数量有限；
 - 角色职责难以清晰划分；
 - 需要降低系统复杂度。
- 考虑多 Agent
- 当：
 - 角色职责明显不同；
 - 不同角色需要不同权限；
 - 任务可独立并行；
 - 需要多个专业能力；
 - 审核和执行必须分离。

设计原则：多 Agent 不是越多越好，复杂度必须与收益匹配。

智能会议助手案例

- 业务目标：从会议纪要中自动提取任务并跟踪执行。
- 工作流程：
- 字段包括：任务名称；任务描述；负责人；截止时间；优先级；当前状态；缺失信息。



实践任务5:设计一个智能会议助手

- 实践要求：智能会议助手能

- 读取会议纪要；
- 提取待办事项；
- 判断负责人和截止时间；
- 生成任务列表；
- 检查是否存在缺失信息；
- 输出结构化结果。

- 参考代码：https://github.com/bettermorn/AIAgent/tree/main/code/MultiAgent/Meeting_Assistant

- 思考问题

- 负责人缺失时如何处理？
- 截止时间不明确时如何追问？
- 是否可以自动发送任务通知？
- 发送通知前是否需要用户确认？

模块六：Agent评估、安全与部署

理解：Agent 不仅要“能运行”，还要做到可靠、安全、可维护。

为什么要评估Agent

- Agent “能够运行”不代表“可以上线”。
- 需要评估：
 - 回答是否准确；
 - 工具是否选择正确；
 - 任务是否完成；
 - 是否会编造信息；
 - 是否存在越权行为； 成本和响应速度是否可接受；
 - 用户是否愿意使用。

Agent评估指标

- 效果指标

- 回答准确率；
- 任务完成率；
- 工具调用正确率；
- 知识检索命中率；
- 幻觉率；
- 格式遵循率。

- 性能指标

- 平均响应时间
- Token 消耗；
- API 调用次数；
- 失败率；
- 并发能力。

- 用户体验指标

- 用户满意度；
- 追问次数；
- 人工接管率；
- 任务完成时间。

测试用例设计

测试用例应包含

字段	示例
用户问题	出差住宿费用标准是什么？
预期答案	根据制度文档返回标准
是否调用工具	是
正确工具	企业知识库
风险等级	低
允许偏差	表述不同但含义一致

- 测试类型
 - 正常问题；
 - 模糊问题；
 - 边界问题；
 - 无答案问题；
 - 工具异常；
 - 越权问题；
 - 恶意 Prompt。

Agent的主要安全风险

- Prompt Injection: 用户诱导 Agent 忽略原有规则。
- 数据泄露: 输出员工、客户、财务等敏感信息。
- 越权访问: 用户访问不属于自己的数据或功能。
- 恶意工具调用: Agent 被诱导执行危险操作。
- 错误业务执行: 错误审批、错误付款或错误删除数据。
- 知识库污染: 恶意或错误的文档进入知识库。

Agent安全措施

- 最小权限原则；
- 工具权限分级；
- 读写工具分离；
- 敏感操作人工确认；
- 参数合法性校验；
- 用户身份认证；
- 数据脱敏；
- 日志记录；
- 调用次数限制；
- 超时和重试；
- 内容审核；
- 高风险任务人工接管。
- **核心原则**：Agent 可以建议，但高风险操作必须经过授权。

Agent部署方式

- 快速演示
 - Jupyter Notebook;
 - Streamlit;
 - Gradio。
- 服务化部署
 - FastAPI;
 - Docker;
 - 云函数;
 - API 网关;
 - 企业内部应用平台。
- 生产环境配套
 - 日志;
 - 监控;
 - 链路追踪;
 - 权限管理;
 - 成本统计;
 - 版本管理;
 - 灰度发布;
 - 故障告警。

上线前检查清单

- 功能

- 是否能完成核心任务?
- 工具调用是否正确?
- 异常是否有提示?
- 是否支持必要的追问?

- 数据

- 知识库内容是否准确?
- 文档是否有版本管理?
- 是否存在敏感信息?

- 安全

- 是否进行权限校验?
- 是否限制高风险操作?
- 是否保留操作日志?

- 性能

- 响应时间是否可接受?
- 成本是否可控?
- 是否支持并发访问?

实践练习4:测试用例

- 为自己的Agent设计测试用例
 - 5 条测试用例;
 - 3 条安全规则;
 - 1 个异常处理方案;
 - 1 个上线前检查清单。
- 参考：会议助手的安全检查 https://github.com/bettermorn/AIAgent/blob/main/code/MultiAgent/Meeting_Assistant/Security_check.md

模块七：综合实战与课程总结

综合项目：企业智能知识助手

- 项目目标: 构建一个能够回答企业问题并调用外部工具的智能助手。
- 最低功能要求
 - 回答企业制度或产品文档问题；
 - 调用至少一个外部工具；
 - 能够判断问题类型；
 - 无答案时不编造；
 - 高风险操作需要确认；
 - 尽可能提供回答来源。
- 参考代码：
 - https://github.com/yh-yao/super_agent_book/tree/main/RAG_%E6%99%BA%E8%83%BD%E5%AE%A2%E6%9C%8D%E4%B8%8E%E7%9F%A5%E8%AF%86%E9%97%AE%E7%AD%94
 - 修改部分参考 <https://github.com/bettermorn/AIAgent/tree/main/code/CSAIAgent>

基础版

- 企业制度问答；
- 计算器工具；
- 对话上下文记忆。

进阶版

- 企业知识库检索；
- 天气查询；
- 数据库查询；
- 生成结构化报告；
- 多轮对话；
- 人工确认节点。

实战流程

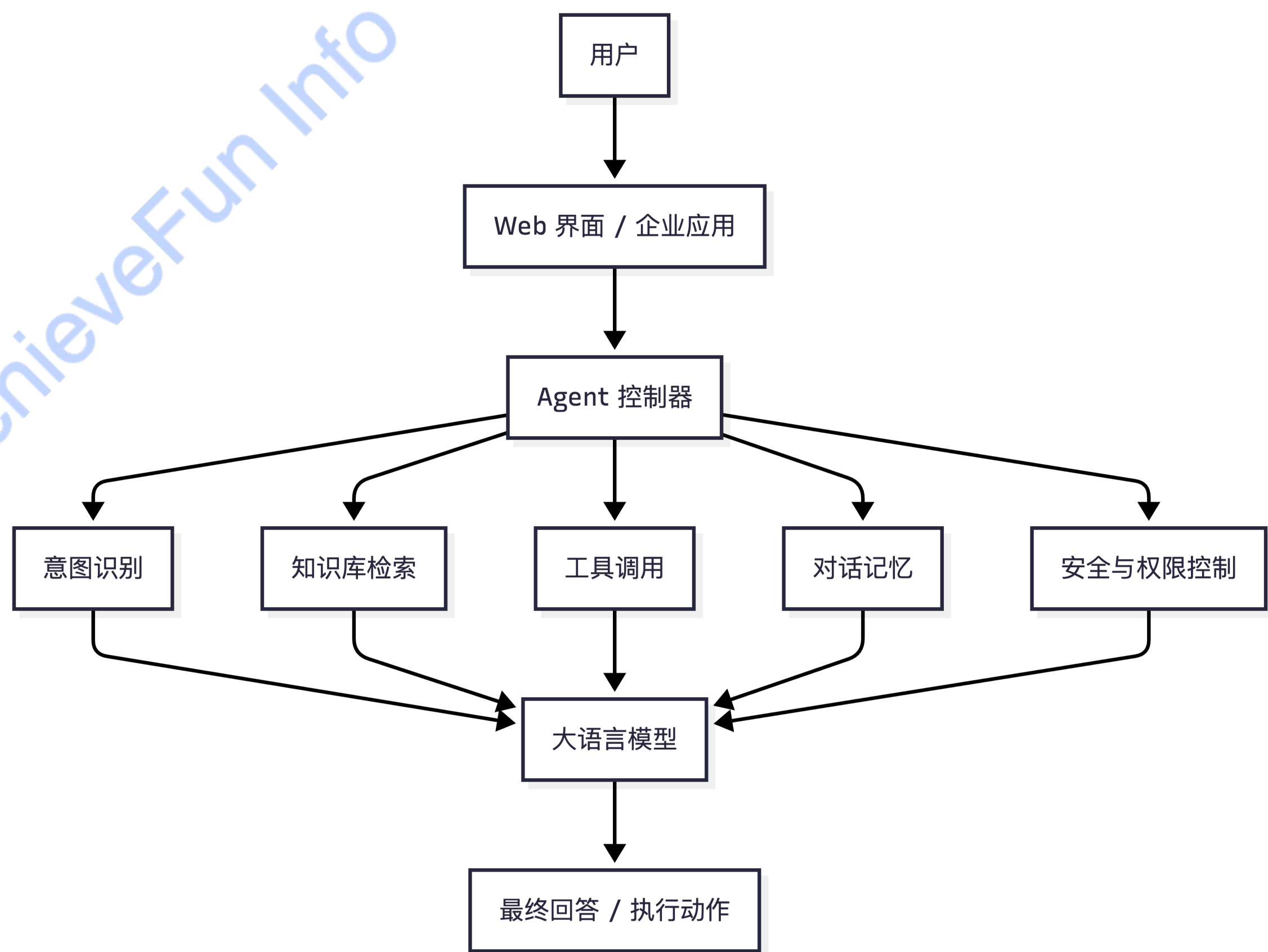
1. 明确业务场景与目标用户
2. 设计 Agent 角色
3. 设计 System Prompt
4. 准备知识库文档
5. 配置工具
6. 实现 Agent 流程
7. 设计异常处理
8. 使用测试问题验证效果
9. 优化 Prompt 和工具描述

Shanghai AchieveFun Info

项目架构示例



Agent 控制器对各功能模块的统一编排



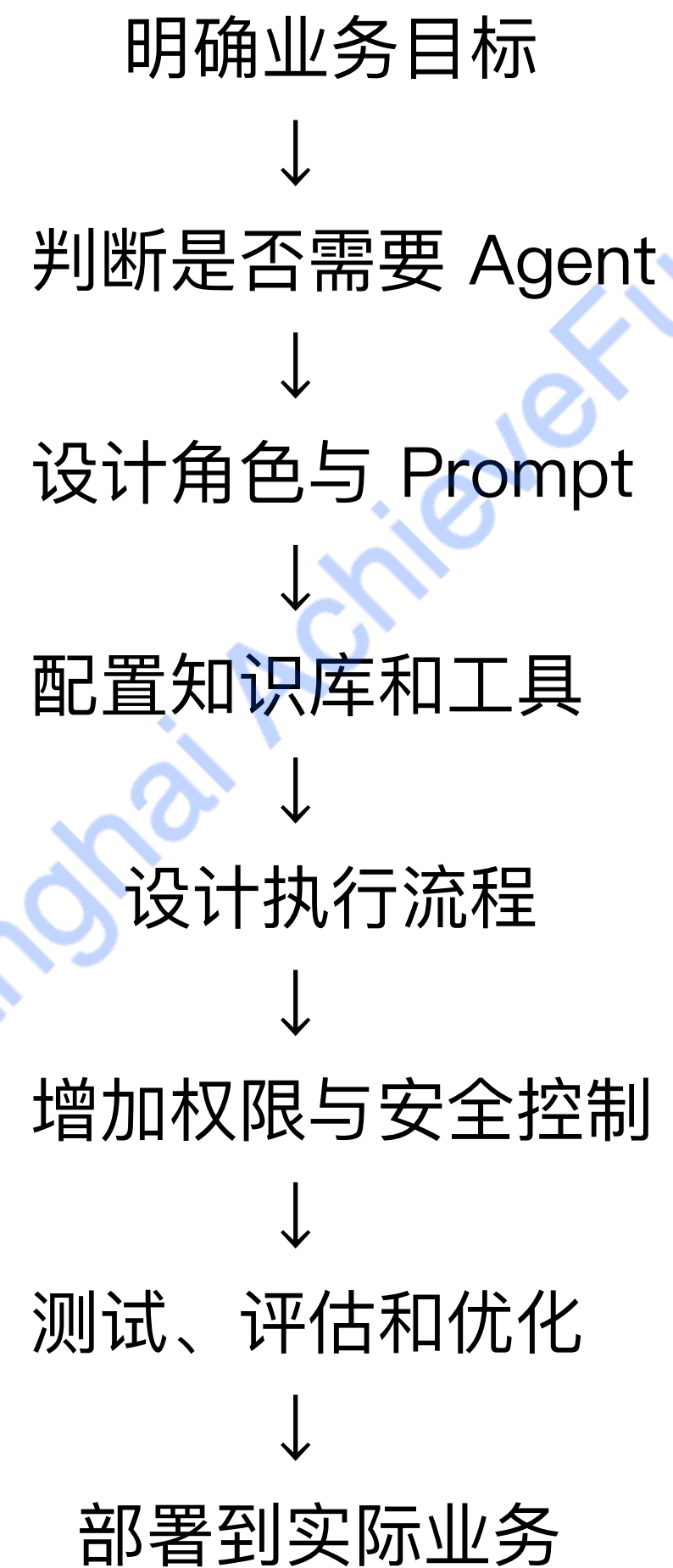
控制器并行协调多个模块

项目展示要求

1. 解决什么业务问题；
2. 目标用户是谁；
3. Agent 的角色是什么；
4. 配置了哪些工具；
5. 是否使用知识库；
6. Agent 如何进行决策；
7. 演示一个完整案例；
8. 当前存在的问题；
9. 后续优化计划。

Shanghai AchieveFun Info

课程核心方法总结



模块	主题	时长 分钟	教学目标	主要内容	实践活动 / 阶段产出
一	AI Agent 认知与应用场景	30	建立对 AI Agent 的整体认识，理解其应用价值	1. AI Agent 的定义与特点 2. Agent 与聊天机器人、Prompt、RAG、自动化工作流的区别 3. Agent 的典型应用场景 4. 判断业务是否适合使用 Agent	分析出差助手、客服、数据分析等案例；能够判断业务需求是否适合采用 Agent
二	Agent 核心架构与运行机制	45	理解 Agent 的技术组成和基本运行循环	1. Agent 基本架构 2. 大语言模型、Prompt、工具、记忆、知识库等核心组件 3. Planning、ReAct、状态机等执行方式 4. Agent 的思考—行动—观察循环 5. Agent 设计原则	将“出差申请助手”拆解为用户输入、知识、工具、决策步骤和输出结果
三	构建第一个 AI Agent	75	使用 Python 和大模型 API 构建可运行的基础 Agent	1. API Key、模型参数、Token、Temperature 2. System Prompt 设计 3. 工具定义与调用 4. Agent 判断是否调用工具 5. 异常处理与结果生成6. LangChain、LangGraph 等框架简介	实现天气查询与计算器 Agent；完成能够理解问题、调用工具并生成回答的基础 Agent
四	RAG 知识库 Agent	60	掌握企业知识库问答的基本方法，降低模型幻觉	1. RAG 工作原理 2. 文档加载与切分 3. 文本向量化 4. 向量数据库与相似度检索 5. 检索结果与模型生成结合	基于企业制度或产品文档搭建知识库；实现文档检索、依据文档回答和来源展示
五	Agent 工作流、工具调用、MCP与多 Agent	60	掌握复杂任务的流程编排和多 Agent 协作方法	1. 固定工作流与 Agent 的区别 2. 顺序、分支、循环、并行、重试流程 3. 人工确认节点 4. 多 Agent 角色设计与协作 5. 单 Agent 与多 Agent 的选择原则	设计“智能会议助手”；实现会议纪要提取、任务分配、负责人识别和结果审核
六	Agent 评估、安全与部署	45	让 Agent 具备可靠、安全和可上线的基础条件	1. 效果、性能和用户体验指标 2. 测试集与评估方法 3. Prompt Injection、数据泄露、越权访问等风险4. 权限控制、人工确认、参数校验、日志记录 5. Streamlit、Gradio、FastAPI、Docker 等部署方式	为项目设计 5 条测试用例、3 条安全规则、1 套异常处理方案和上线检查清单

关键知识点回顾

- **AI Agent**

- 能理解目标；
- 能动态规划；
- 能调用工具；
- 能根据反馈调整

- **RAG**

- 先检索，再生成；
- 适合企业私有知识；
- 需要关注切分、召回和来源。

- **工具调用**

- 工具职责要单一；
- 参数定义要清晰；
- 高风险操作需要确认。

- **Agent 工程化**

- 必须测试；
- 必须控制权限；
- 必须记录日志；
- 必须关注成本和性能。

让AI Agent真正解决业务问题

- 思考：你所在的业务中，哪一个任务最适合由 AI Agent 完成？
- 设计提示
 - 目标是什么？
 - 需要哪些知识？
 - 需要调用哪些工具？
 - 哪些环节需要人工确认？
 - 如何评估效果？
 - 上线后如何保障安全？

学习参考书籍和代码

- 黄佳 新手入门 <https://github.com/huangjia2019/ai-agents/tree/main>
- 黄佳 设计AI Agent <https://github.com/huangjia2019/designing-ai-agents/>
- 黄佳 知识库 <https://github.com/huangjia2019/ai-knowledge-base>
- 姚宇航 从零构建AI Agent：大模型驱动的智能体设计与实战 https://github.com/yh-yao/super_agent_book



深度契合头部企业工程需求

